10.02.2026

Командная строка. Решение конечных уравнений. Решение СЛАУ. Вычислительная геометрия.

Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

Новосибирский государственный университет

*НАСТОЯЩАЯ НАУКА

Давайте познакомимся

Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.

Новосибирский государственный университет
*НАСТОЯЩАЯ НАУКА

План лекции

Командная строка	Решение конечных уравнений	Решение СЛАУ	Вычислительная геометрия
10 минут	15 минут	35 минут	30 минут

План лекции

Командная строка	Решение конечных уравнений	Решение СЛАУ	Вычислительная геометрия
10 минут	20 минут	40 минут	20 минут

Аргументы командной строки

До появления современного графического интерфейса существовал интерфейс командной строки. Примерами могут служить DOS и Unix, к тому же терминал Linux предоставляет Unix-подобную среду командной строки. **Командная строка** — это место, где вы вводите с клавиатуры информацию для запуска своей программы в среде командной строки.

Пример для программы:

$ fuss -r Ginger

Аргументы командной строки - демо

repeat.c
int main(int argc, char *argv[])
{

}

Запуск программы

repeat My name is Mikhail

argv[0] argv[1] argv[2] argv[3]

argc = 4

4 строки

```
PowerPoint/lection_x_2$ gcc main.c -o repeat
mikhail@DESKTOP-R6I9BAG:/mnt/c/Users/Mfili/Desktop/lea
PowerPoint/lection_x_2$ ./repeat My name is Mikhail
Количество аргументов, указанных в командной строке: 4
1: My
2: name
3: is
4: Mikhail
```

```c
/* repeat.c -- функция main() с аргументами */
#include <stdio.h>
int main(int argc, char *argv[])
{
    int count;
    printf("Количество аргументов, указанных в командной строке: %d\n", argc - 1);
    for (count = 1; count < argc; count++)
        printf("%d: %s\n", count, argv[count]);
    printf("\n");
    return 0;
}
```

Преобразования строк в числа

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[])
{
    int i, times;
    if (argc != 2 || (times = atoi(argv[1])) < 1)
        printf("Использование: %s положительное-число\n", argv[0]);
    else
        for (i = 0; i < times; i++)
            puts("Хорошего дня!");
    return 0;
}
// $ gcc main.c -o main
// $ ./main 3
// Хорошего дня!
// Хорошего дня!
// Хорошего дня!
```

План лекции

Командная строка	Решение конечных уравнений	Решение СЛАУ	Вычислительная геометрия
10 минут	20 минут	40 минут	20 минут

Введение

Решение большинства физических и математических задач приводит к возникновению конечных уравнений, которые могут быть представлены в форме $f(x) = 0$. Если $f(x)$ является полиномом, такие уравнения называют ***алгебраическими***, в противном случае — ***трансцендентными***. Зачастую точное решение конечных уравнений не может быть выражено через элементарные функции. Примером может служить задача из квантовой механики о поиске уровня энергии E основного состояния в прямоугольной яме конечной глубины:

$$-\frac{\hbar^2}{2m}\psi'' + (U(x) - E)\psi = 0, \qquad U(x) = \begin{cases} -U_0, |x| \le a \\ 0, |x| > a \end{cases}$$

Волновая функция основного состояния чётная и имеет вид

$$\psi(x) = \begin{cases} A\cos kx, |x| \le a \\ Be^{-qx}, |x| > a \end{cases}, \qquad k^2 = \frac{2m}{\hbar^2}(U_0 + E), \qquad q^2 = -\frac{2m}{\hbar^2}E$$

Условие непрерывности решения $\psi(x)$ и его первой производной в точке $x = a$ приводит к трансцендентному уравнению, точное решение которого не может быть представлено в элементарных функциях:

$$ctg\sqrt{\frac{2ma^2U_0}{\hbar^2}(1-\xi)} = \sqrt{\frac{1}{\xi} - 1}, \qquad \xi = -\frac{E}{U_0}$$

Метод деления отрезка пополам

Пусть на отрезке $[a, b]$ задана непрерывная функция $f : [a, b]$, причём $f(a) \cdot f(b) \leq 0$. Непрерывная функция принимает на отрезке все промежуточные значения и, следовательно, существует точка $x^* \in [a, b] : f(x^*) = 0$. Наша задача состоит в нахождении такой точки (корня $f(x)$).

Для решения поставленной задачи вычислим значение функции f в средней точке отрезка, $f\ (1/2(a + b))$. Если $f\ (a) \cdot f(1/2\ (a + b)) \leq 0$, положим $a_1 = a$, $b_1 = 1/2\ (a + b)$, в противном случае возьмем $a_1 = 1/2\ (a + b)$, $b_1 = b$. Шаг дихотомии выполнен. Мы свели задачу о поиске корня на отрезке $[a, b]$ к аналогичной задаче для отрезка $[a_1, b_1]$, ширина которого в два раза меньше. Повторяя процесс многократно, получим последовательность отрезков $[a_n, b_n]$, причём $b_n - a_n \to 0$ при $n \to \infty$.

Максимальная абсолютная погрешность определения корня x^* равна $1/2\ (b - a)$ и на каждой итерации уменьшается вдвое. Следовательно, если искомое значение x^* известно по порядку величины, каждый шаг дихотомии даёт один дополнительный бит мантиссы. Это позволяет оценить необходимое число шагов для достижения требуемой точности:

$$N \approx \log_2 {}^{\delta_0}/_{\delta}$$

где δ_0 и δ — точность начального приближения и требуемая точность нахождения корня соответственно.

Метод простых итераций

$f(x) = 0$ перепишем в виде $\phi(x) = x$, где $\phi(x) := f(x) + x$.

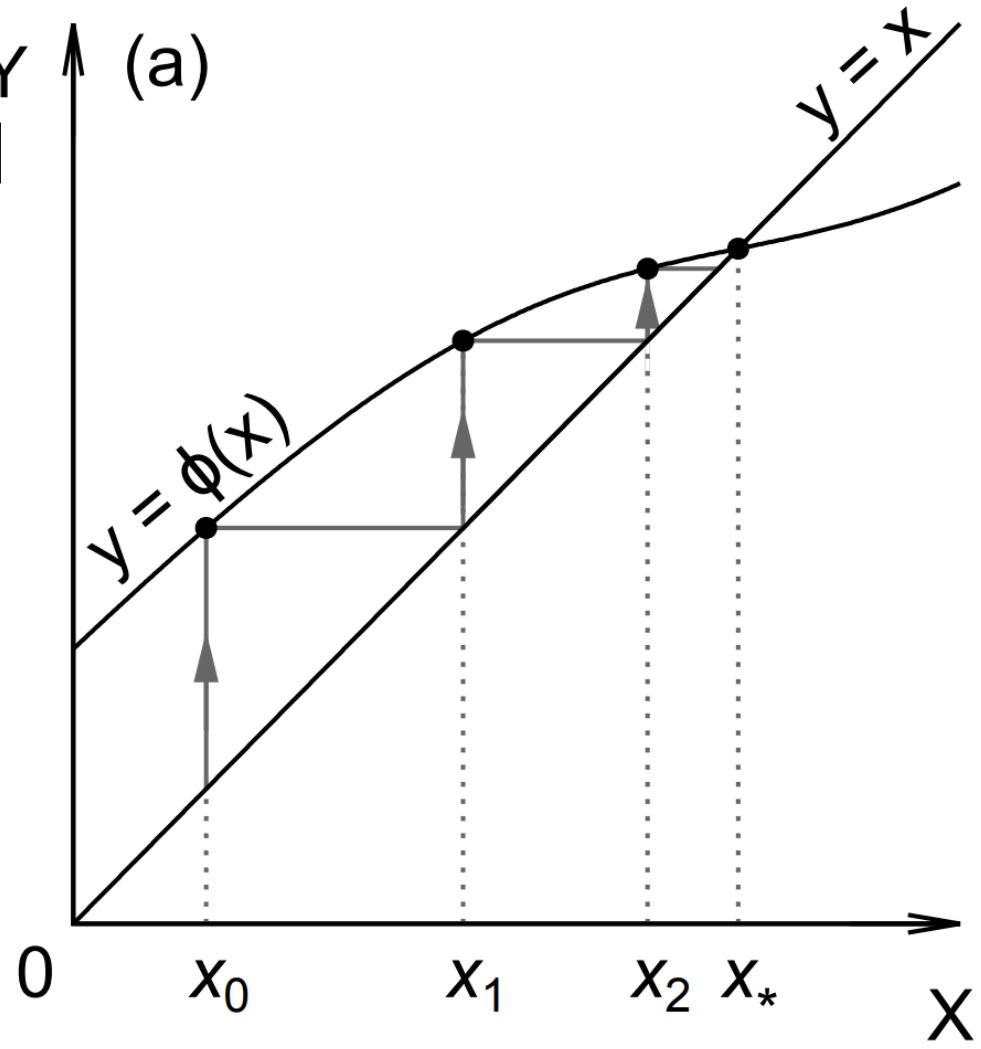
Пусть функция $\phi(x)$ гладкая и удовлетворяет условию $\phi'(x) \le q$, где $q < 1$ — некоторая постоянная.

Пусть x_0 начальное приближение к корню x^*. Организуем итерационный процесс вида

$x_{n+1} = \phi(x_n)$, $\quad n = 0, 1, 2, \ldots$

Покажем, что предел последовательности x_n существует и равен искомому корню x^*: $\phi(x^*) = x^*$.

Заметим, $\phi(x)$ реализует сжимающее отображение, уменьшая длину произвольного отрезка $[a, b] \to [\phi(a), \phi(b)]$ минимум в $1/q$ раз в силу ограниченности производной $|\phi'| \le q$. При этом корень x^* по определению отображается функцией $\phi(x)$ сам в себя. Следовательно, выбирая $a = x_0$ и $b = x^*$, имеем $|x_n - x^*| \to 0$ при $n \to \infty$, т. е. $x_n \to x^*$, ч. т. д. ■



Оценим количество итераций, необходимых для получения приближённого значения корня с требуемой точностью δ. Пусть $\phi(x)$ гладкая, а начальное приближение x_0 достаточно точным, так что $\phi(x_0) = \phi(x^*) + \phi'(x^*)(x_0 - x^*)$. Учитывая, что $\phi(x^*) = x^*$, получаем $(x_1 - x^*) = (x_0 - x^*)\,\phi'(x^*)$, т. е. абсолютная погрешность определения искомого корня умножается на каждой итерации на величину $|\phi'| \le q < 1$.

Метод простых итераций

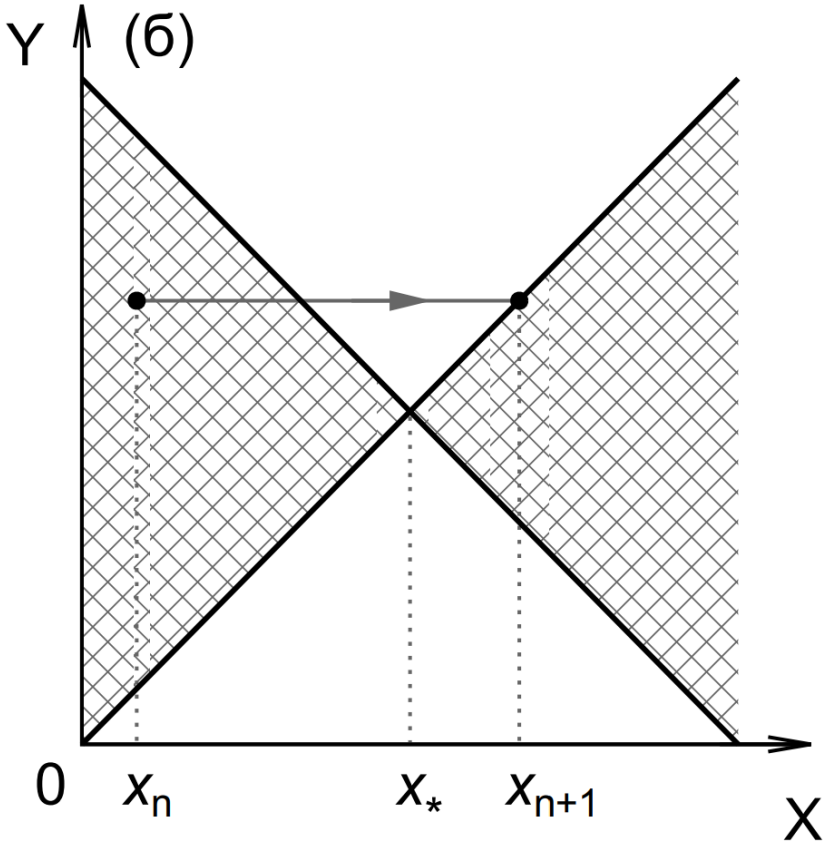
Получаем оценку для числа итераций:

$$N \approx \frac{\log_2 {}^{\delta_0}/_{\delta}}{-\log|\varphi'|}$$

Метод простых итераций сходится быстрее метода деления отрезка пополам при условии $|\phi'(x^*)| < 1/2$. Очевидно, что проведённое выше рассмотрение и оценка количества итераций в последнем случае будут неприменимы: при малых $|\phi'(x^*)|$ в разложении $\phi(x_0) \approx \phi(x^*)+\phi'(x^*)\,(x_0 - x)$

Заметим, что сформулированное вначале ограничение $|\phi'| \le q < 1$ является достаточным, но не необходимым условием сходимости итерационного процесса.

Для сходимости итераций достаточно, чтобы отображение ϕ уменьшало длину произвольного отрезка, один из концов которого совпадает с искомым корнем x^*, при этом производная ϕ' может принимать сколь угодно большие значения или даже вовсе не существовать.

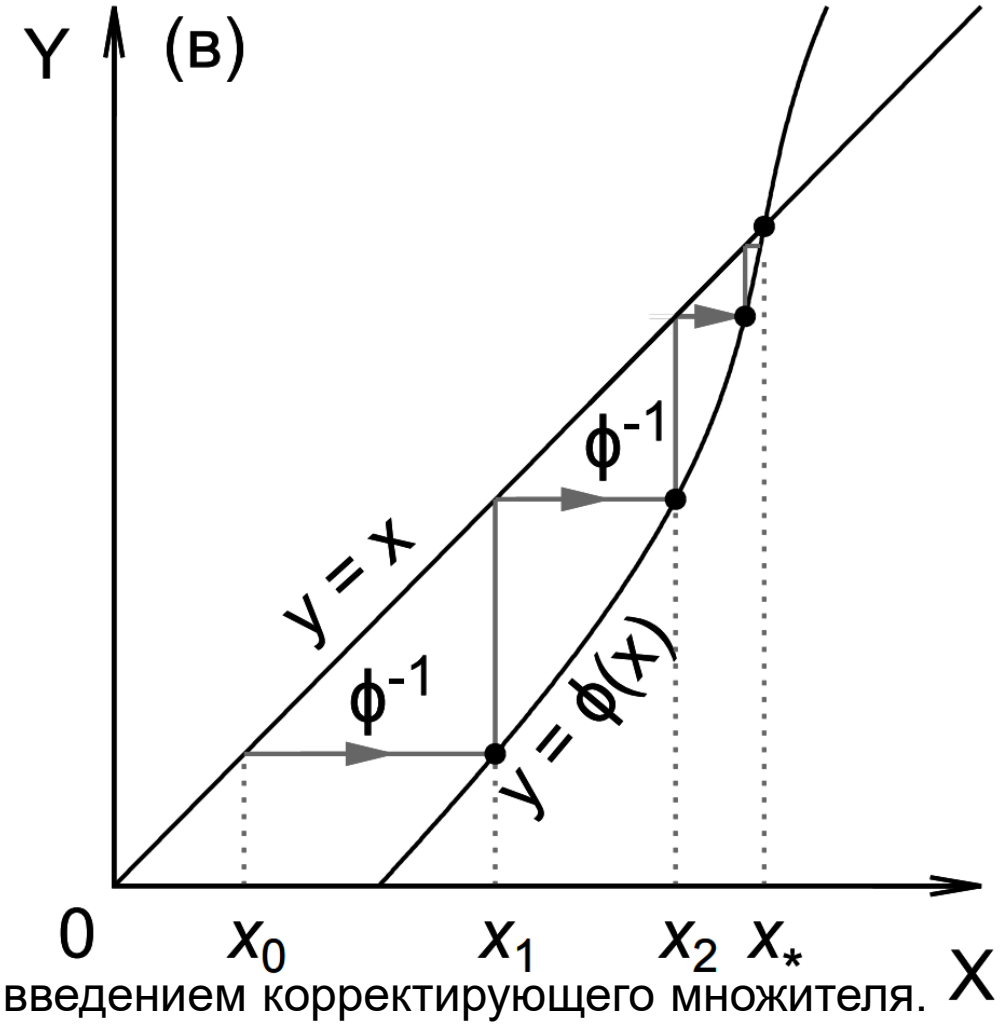


Итерационный процесс будет сходиться при любом начальном приближении x_0, если график функции $\phi(x)$ лежит между прямыми $y_1 = x^* + q(x - x^*)$ и $y_2 = x^* - q(x - x^*)$, где $0 < q < 1$ — некоторая постоянная. На рисунке видно, что итерация $x_{n+1} = \phi(x_n)$ из любой точки $(x_n, \phi(x_n))$ внутри заштрихованных секторов улучшит приближение к корню: $|x_{n+1} - x^*| < |x_n - x^*|$. Очевидно, многие функции не удовлетворяют даже более слабому условию.

Метод простых итераций

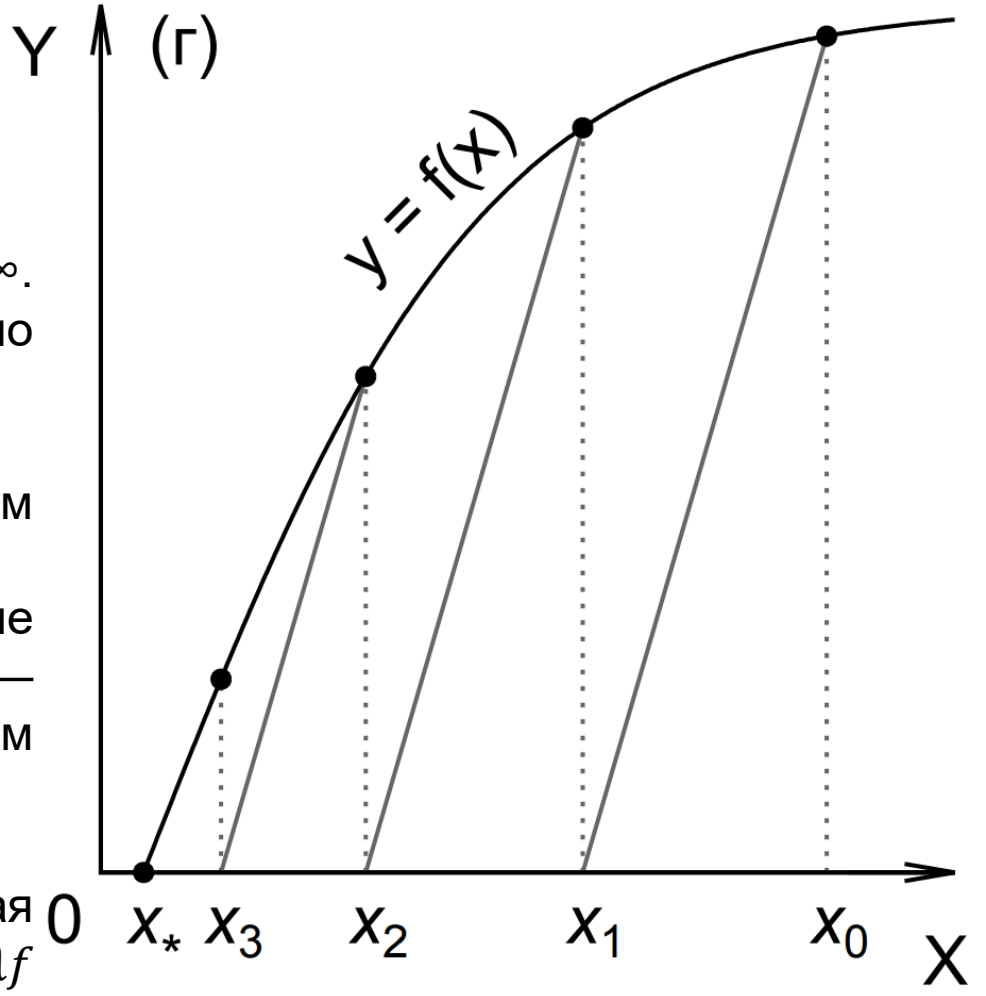
Первая возможность связана с переходом от функции ϕ к обратной функции ϕ^{-1}.

В случае, если $|\phi'(x)| \geq q > 1$, можно перейти к обратным функциям в уравнении $\phi(x) = x$, в результате получим итерационный процесс $x_{n+1} = \phi^{-1}(x_n)$, который будет сходиться в силу соотношения $(\phi^{-1})' = 1/\phi'$. Геометрический смысл итераций для обратной функции показан на рисунке, стрелками обозначены вычисления функции $\phi^{-1}(x)$. Производная $\phi' > 1$, поэтому итерационный процесс $x_{n+1} = \phi(x_n)$ будет расходящимся. Для сходимости необходимо, по сути, двигаться в обратную сторону, что и достигается переходом к вычислению обратной функции $\phi^{-1}(x_n)$ вместо $\phi(x_n)$. Итерационный процесс для обратных функций целесообразно использовать в случае, когда обратная функция ϕ^{-1} может быть относительно легко вычислена, а $|\phi'(x^*)| >> 1$.



Другая модификация метода итераций может быть получена введением корректирующего множителя. Вместо уравнения $x = x + f(x) = \phi(x)$ будем решать уравнение $x = x - \lambda f(x)$, множество решений которого, очевидно, совпадает с множеством корней уравнения $f(x) = 0$ при условии $\lambda = 0$.

Метод простых итераций



Пусть функция $f(x)$ непрерывно дифференцируема, так что $|f'(x)| < \infty$. Производная $\tilde{\phi}'(x) = 1 - \lambda f'(x)$ может быть сделана сколь угодно малой в точке $x = x^*$, что обеспечит сходимость

$$x_{n+1} = x_n - \lambda f(x_n)$$

для любой гладкой функции $f(x)$. Знак λ нужно выбирать равным знаку производной, а значение — по возможности ближе к $1/f'(x^*)$.

Выход из цикла итераций можно осуществлять, проверяя условие малости изменения x на последней итерации: $|x_n - x_{n-1}| < \delta$, где δ — требуемая точность. Однако данная оценка может быть слишком грубой, особенно в случае $|\phi'(x^*)| \approx 1$.

Критерий завершения итерационного процесса

Рассмотрим последовательность $\{x_k\}$, сходящуюся к корню x^*, полагая x_0 достаточно близким к x^*. Обозначая $R_n := x^* - x_n$, из $x_{n+1} = x_n - \lambda f(x_n)$ получаем

$$R_{n+1} \approx 1 - \lambda f'(x*) R_n \equiv qR_n,$$

т. е. итерации сходятся приблизительно как сумма геометрической прогрессии со знаменателем $q = (x_n - x_{n-1})/(x_{n-1} - x_{n-2})$. Итерационный процесс может быть остановлен, когда погрешность R_n текущего приближения к корню не превосходит требуемой точности вычислений δ.

Поскольку $x_n - x_{n-1} = R_{n-1} - R_n \approx R_n/q - R_n$, приходим к условию

$$|R_n| \approx \left|\frac{x_n - x_{n-1}}{\frac{1}{q} - 1}\right| = \frac{(x_n - x_{n-1})^2}{|2x_{n-1} - x_n - x_{n-2}|} < \delta$$

Метод Ньютона — Рафсона

Скорость сходимости можно существенно повысить, положив $\lambda(x)$ = $1/f'(x)$ с тем, чтобы наклон был равен производной функции $f(x)$. При этом мы получим итерационный процесс

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

известный как метод Ньютона, или Ньютона — Рафсона.

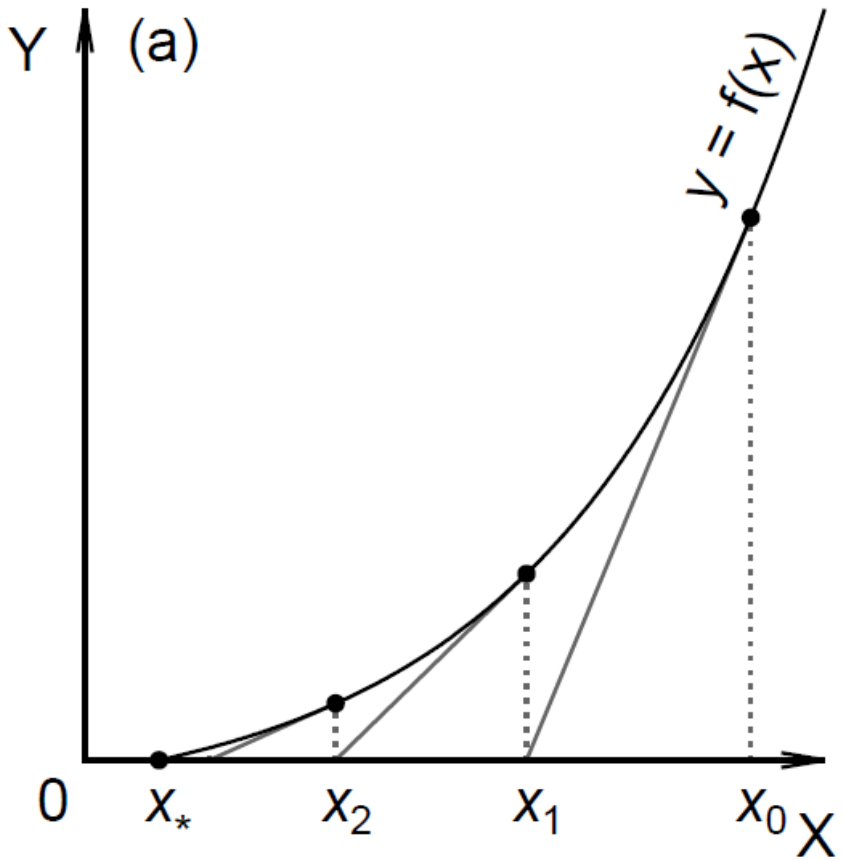
В выражении есть точное решение линеаризованной задачи $f(x)$ = 0.

Исследуем скорость сходимости процесса к корню x^*. Предположим, что нам задано достаточно хорошее приближение x_n,

$f(x_n) \approx f'(x^*)\,(x_n - x^*) + 1/2 f''(x^*)\,(x_n - x^*)^2$, а $f'(x_n) \approx f'(x^*) + f''(x^*)\,(x_n - x^*)$.

Вычитая x^* из обеих частей уравнения, имеем:

$$R_{n+1} \approx R_n + \frac{f'(x_*)(-R_n) + \frac{1}{2}f''(x_*)(-R_n)^2}{f'(x_*) - R_n f''(x_*)}$$



В методе Ньютона погрешность на каждом следующем шаге оказывается квадратично мала по сравнению с погрешностью на предыдущем шаге, $R_{n+1} = \alpha R^2_n$.

Выражение допускает возможность расходящегося процесса $R_{n+1} > R_n$ при достаточно больших R_n. Другими словами, если начальное приближение x_0 лежит в области $a < x_0 < b$, так что x^* (a, b), процесс сходится к корню x^*, в противном случае итерации расходятся (либо сходятся к другому корню вне $[a, b]$).

Метод секущих

Зачастую, однако, производная функции $f(x)$ неизвестна, вместо $f'(x)$ при вычислении корректирующего множителя $\lambda = 1/f'(x)$ можно использовать асимптотические разложения $f(x)$ или вычисленные с помощью разделённой разности по результатам двух последних вычислений $f\ (x)$:

$$x_{x+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}, n = 1,2, \ldots$$

Итерационный процесс является двухшаговым: для нахождения следующего приближения x_{n+1} необходимо знать два предыдущих.

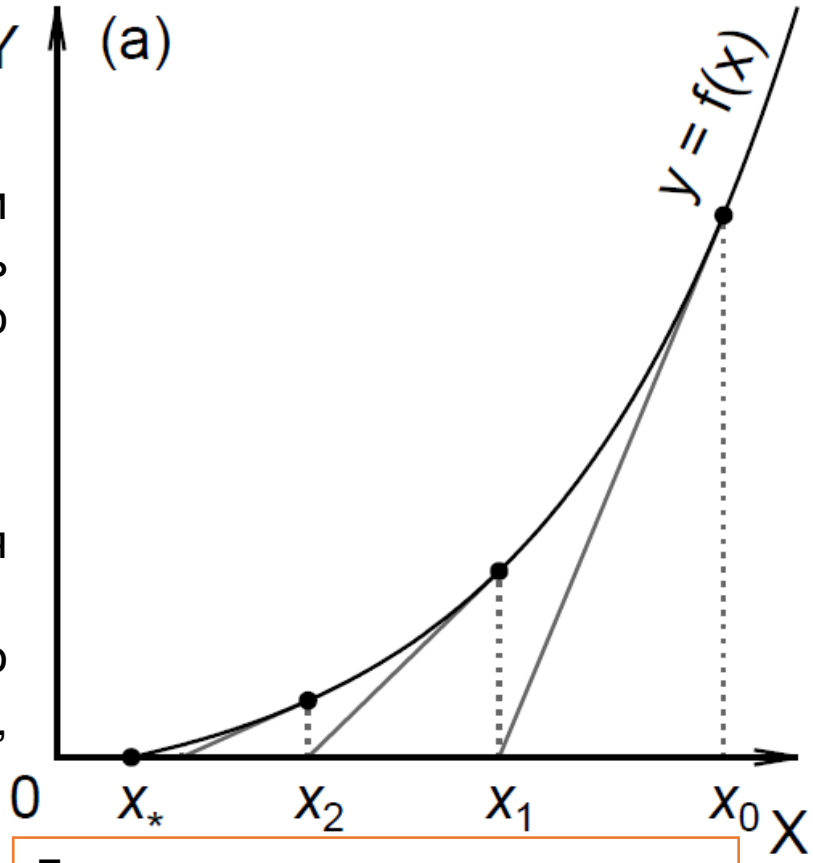
Скорость сходимости процесса. Полагая x_n и x_{n-1} достаточно близкими к корню x^* и заменяя $f\ (x) \approx f\ '(x^*)\ (x - x^*) + 1/2\ f''(x^*)(x - x^*)^2$, имеем:

$$R_{n+1} \approx R_n - R_n\frac{1 - aR_n}{1 - a(R_n + R_{n-1})}, R_n = x_* - x_n, a = \frac{f''(x_*)}{f'(x_*)}$$

Удерживая члены не выше второго порядка по R_n и R_{n-1}, получаем:

$$R_{n+1} \approx aR_nR_{n-1}$$

Попробуем найти степенной закон убывания погрешности на каждой итерации. Подставляя $|R_{n+1}| = a^\alpha|R_n|^\beta$, получаем два условия на показатели α и β: $\alpha\beta = 1$, $\beta^2 - \beta - 1 = 0$. Для сходимости итерационного процесса необходимо $\beta > 0$, откуда $\beta = \frac{1}{2}(1 + \sqrt{5}) \approx 1.618$.



Двукратное вычисление значения функции f позволяет сделать два шага методом секущих $|R_{n+2}| \propto |R_n|^{\beta^2}$. Поскольку $\beta^2 \approx 2{,}618$, метод секущих может сходиться быстрее метода Ньютона.

Многомерное обобщение

Метод Ньютона очевидным образом может быть обобщён на многомерный случай. Пусть задана система уравнений f (x) = 0, где f = $\{f_1, \ldots, f_n\}$, x = $\{x_1, \ldots, x_n\}$. Выбрав некоторое начальное приближение $x^{(0)} = \{x_1^{(0)}, \ldots, x_n^{(0)}\}$ к корню $x^{(*)} = \{x_1^{(*)}, \ldots, x_n^{(*)}\}$, построим последовательность $x^{(n)}$, n = 1, 2, . . ., сходящуюся к искомому корню. Для этого, как и в одномерном случае, линеаризуем уравнения f(x) = 0. Используя тензорные обозначения и предполагая суммирование по повторяющимся индексам, имеем:

$$0 = f\left(x^{(n)} + \delta x^{(n)}\right) \approx f_i(x^{(n)}) + J_{ij}^{(n)} \delta x_j^{(n)}, \text{где} J_{ij}^{(n)} = \left.\frac{\partial f_i}{\partial x_j}\right|_{x=x^{(n)}}$$

откуда получаем для $x^{(n+1)} = x^{(n)} + \delta x^{(n)}$:

$$x_i^{(n+1)} = x_i^{(n)} - \left[\left(J^{(n)}\right)^{-1}\right]_{ij} f_i(x^{(n)})$$

Поиск комплексных корней (доп. слайд)

В некоторых задачах может возникнуть необходимость поиска нулей функции на комплексной плоскости. Пусть $f\ (z)$ — аналитическая функция в области $\mathcal{D}$, и $|f(z)| < \infty\ \forall z \in \mathcal{D}$. Разложим $f(z)$ и её производную $f'(z)$ в ряд в точке $z_0 \in \mathcal{D}$:

$$f\ (z) = \sum_{k=n}^{\infty} \frac{c_k}{k!}\ (z\ -\ z_0)^k\ , f'(x) = \sum_{k=1}^{\infty} \frac{c_k}{(k-1)!}\ (z\ -\ z_0)^{k-1}\ , c_k = f^{(k)}(z_0)$$

Если z_0 — нуль функции f, то $c_0 = 0$ и $f'/f = (z - z_0)^{-1} + O(|z - z_0|^0)$. Если z_0 — нуль кратности n, то

$$f\ (z) = \sum_{k=n}^{\infty} \frac{c_k}{k!}\ (z\ -\ z_0)^k\ , \frac{f'(\mathrm{z})}{f(\mathrm{z})}\ =\ \frac{\mathrm{n}}{z - z_0} + \mathcal{O}\ (|\mathrm{z}\ -\ \mathrm{z}_0|^0).$$

Используя интегральную формулу Коши, имеем $\oint_\gamma \frac{f'(\mathrm{z})}{f(\mathrm{z})} dz = 2\pi i \sum_i n_j = 2\pi i N$

С точностью до множителя $2\pi i$ получили количество нулей функции $f(z)$ внутри контура γ с учетом их кратности.
Стратегия поиска комплексных корней $f\ (z)$ с использованием может быть следующей. Выбираем контур интегрирования γ так, чтобы внутрь него попадало 1-2 нуля функции $f\ (z)$. Допустим, для примера, $\sum n_j = 2$. Далее вычисляем интегралы вида

$$I_1 = \frac{1}{2\pi i} \oint_\gamma \frac{f'(\mathrm{z})}{f(\mathrm{z})} z dz = \sum_i z_j n_j = z_1 + z_2; \quad I_2 = \frac{1}{2\pi i} \oint_\gamma \frac{f'(\mathrm{z})}{f(\mathrm{z})} z^2 dz = \sum_i z_j^2 n_j = z_1^2 + z_2^2$$

Решение (z_1, z_2) полученной системы уравнений даёт нам решение исходной задачи о нахождении комплексных корней $f(z)$.

Заметим, что задача нахождения комплексных корней может быть решена с использованием рассмотренных выше методов Ньютона, секущих или метода итераций. Эти методы могут быть гораздо более эффективны в случае, если известно достаточно хорошее начальное приближение к искомому корню.

Сравнение методов

Какой из рассмотренных выше методов лучше? Однозначного ответа на этот вопрос нет — оптимальный метод зависит от условий поставленной задачи.

Метод дихотомии

- сравнительно невысокая, но гарантированная скорость сходимости для корней нечётной кратности, тогда как сходимость других рассмотренных выше методов может быть условной.
- может быть использован для получения грубого приближения к корню с последующим его уточнением каким-либо другим методом.

Метод Ньютона

- эффективен, подходит и для корней с чётной кратностью, но требует вычисления производной.

Метод секущих

- эффективен, однако может приводить к «разболтке» и обеспечивать более низкую точность, особенно в случае кратных корней.

Метод простых итераций

- исключительно прост в реализации и при этом может обеспечивать быструю сходимость к корню, особенно в случае $|\phi'(x^*)| << 1$.

Метод Брента

одно из оптимальных решений в случае, когда неизвестны производные функции, объединяет достоинства дихотомии и метода Ньютона. Следующее приближение строится по трём начальным точкам с использованием квадратичной интерполяции $x(y)$, а в случаях, когда найденное с помощью интерполяции приближение не обеспечивает сходимости, применяется бисекция.

План лекции

Командная строка	Решение конечных уравнений	Решение СЛАУ	Вычислительная геометрия
10 минут	20 минут	40 минут	20 минут

СЛАУ

$$\begin{cases} a_{11} \cdot x_1 + a_{12} \cdot x_2 + \cdots + a_{1n} \cdot x_n = b_1 \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 + \cdots + a_{2n} \cdot x_n = b_2 \\ \vdots \\ a_{m1} \cdot x_1 + a_{m2} \cdot x_2 + \cdots + a_{mn} \cdot x_n = b_m \end{cases}$$

Все нужно считать для расширенной матрицы (Ab)

- Чтобы решений не было, нужно – количество линейно-независимых уравнений n > m
- Чтобы решение было единственное, нужно – количество линейно-независимых уравнений n = m
- Чтобы решение было множество, нужно – количество линейно-независимых уравнений n < m

Подробности в курсе линейной алгебры или по ссылке.

Итак, мы можем преобразовать в матричный вид A · X = B.

$$\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

Как понять, что решение 1:

Найти и исключить линейно-независимые уравнения, в итоге определитель квадратной матрицы не равен 0!

Метод Гаусса

Метод Гаусса — классический метод решения системы линейных алгебраических уравнений (СЛАУ).

$$\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_m \end{pmatrix}$$

Метод Гаусса решения системы линейных уравнений включает в себя 2 стадии:

- последовательное (прямое) исключение;
- обратная подстановка.

Метод Гаусса

Последовательное исключение (предположим n ≤ m, иначе множество решений)

$$\begin{cases} a_{11}\cdot x_1 + a_{12}\cdot x_2 + \cdots + a_{1n}\cdot x_n = b_1 \\ a_{21}\cdot x_1 + a_{22}\cdot x_2 + \cdots + a_{2n}\cdot x_n = b_2 \\ \vdots \\ a_{m1}\cdot x_1 + a_{m2}\cdot x_2 + \cdots + a_{mn}\cdot x_n = b_m \end{cases}$$

Делим $i-$ое уравнение на a_{i1}

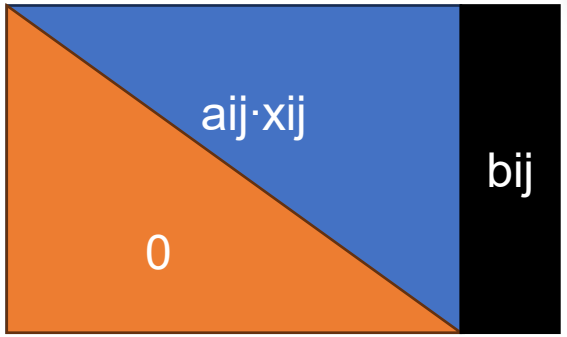
$$\begin{cases} x_1 + \frac{a_{12}}{a_{11}}\cdot x_2 + \cdots + \frac{a_{1n}}{a_{11}}\cdot x_n = \frac{b_1}{a_{11}} \\ x_1 + \frac{a_{22}}{a_{21}}\cdot x_2 + \cdots + \frac{a_{2n}}{a_{21}}\cdot x_n = \frac{b_2}{a_{21}} \\ \vdots \\ x_1 + \frac{a_{m2}}{a_{m1}}\cdot x_2 + \cdots + \frac{a_{mn}}{a_{m1}}\cdot x_n = \frac{b_m}{a_{m1}} \end{cases} \rightarrow \begin{cases} x_1 + \frac{a_{12}}{a_{11}}\cdot x_2 + \cdots + \frac{a_{1n}}{a_{11}}\cdot x_n = \frac{b_1}{a_{11}} \\ 0x_1 + \left(\frac{a_{22}}{a_{21}} - \frac{a_{12}}{a_{11}}\right)\cdot x_2 + \cdots + \left(\frac{a_{2n}}{a_{21}} - \frac{a_{1n}}{a_{11}}\right)\cdot x_n = \frac{b_2}{a_{21}} - \frac{b_1}{a_{11}} \\ \vdots \\ 0x_1 + \left(\frac{a_{m2}}{a_{m1}} - \frac{a_{12}}{a_{11}}\right)\cdot x_2 + \cdots + \left(\frac{a_{mn}}{a_{m1}} - \frac{a_{1n}}{a_{11}}\right)\cdot x_n = \frac{b_m}{a_{m1}} - \frac{b_1}{a_{11}} \end{cases}$$

Отнимаем из каждого уравнения первое

Метод Гаусса

Последовательное исключение

$$\begin{cases} x_1 + \frac{a_{12}}{a_{11}} \cdot x_2 + \cdots + \frac{a_{1n}}{a_{11}} \cdot x_n = \frac{b_1}{a_{11}} \\ 0x_1 + \left(\frac{a_{22}}{a_{21}} - \frac{a_{12}}{a_{11}}\right) \cdot x_2 + \cdots + \left(\frac{a_{2n}}{a_{21}} - \frac{a_{1n}}{a_{11}}\right) \cdot x_n = \frac{b_2}{a_{21}} - \frac{b_1}{a_{11}} \\ \vdots \\ 0x_1 + \left(\frac{a_{m2}}{a_{m1}} - \frac{a_{12}}{a_{11}}\right) \cdot x_2 + \cdots + \left(\frac{a_{mn}}{a_{m1}} - \frac{a_{1n}}{a_{11}}\right) \cdot x_n = \frac{b_m}{a_{m1}} - \frac{b_1}{a_{11}} \end{cases}$$



Замена переменных

$$\begin{cases} x_1 + a'_{12} \cdot x_2 + \cdots + a'_{1n} \cdot x_n = b'_1 \\ 0 + a'_{22} \cdot x_2 + \cdots + a'_{2n} \cdot x_n = b'_2 \\ \vdots \\ 0 + a'_{m2} \cdot x_2 + \cdots + a'_{mn} \cdot x_n = b'_m \end{cases}$$

Теперь остается аналогичная система внутренний квадрат, проделываем аналогичные операции (Предположим, в итоге получилась n уравнений, m-n – оказались нулевыми – линейно зависимыми)

$$\begin{cases} x_1 + a'_{12} \cdot x_2 + \cdots + a'_{1n} \cdot x_n = b'_1 \\ 0 + x_2 + \cdots + a''_{2n} \cdot x_n = b''_2 \\ \vdots \\ 0 + 0 + \cdots + x_n = b_n'^{(n)} \\ 0 + 0 + \cdots + 0 = 0 \\ \vdots \\ 0 + 0 + \cdots + 0 = 0 \end{cases}$$

m-n уравнений

Метод Гаусса

Обратная подстановка

$$\begin{cases} x_1 + a'_{12} \cdot x_2 + \cdots + a'_{1n} \cdot x_n = b'_1 \\ 0 + x_2 + \cdots + a''_{2n} \cdot x_n = b''_2 \\ \vdots \\ 0 + 0 + \cdots + x_n = b_n'^{(n)} \end{cases}$$

Обратная подстановка предполагает подстановку полученного на предыдущем шаге значения переменной x_n в предыдущие уравнения. Эта процедура повторяется для всех оставшихся решений:

$$x_n = b_n'^{(n)}$$
$$x_{n-1} = b_{n-1}'^{(n-1)} - a_{(n-1)n}'^{(n-1)} x_n$$
$$x_{n-2} = b_{n-2}'^{(n-2)} - a_{(n-2)n}'^{(n-2)} x_n - a_{(n-2)(n-1)}'^{(n-2)} x_{n-1}$$
$$\ldots$$
$$x_2 = b_2'' - a_{2n}'' x_n - a''_{2\,(n-1)} x_{n-1} - \cdots - a_{23}'' x_3$$
$$x_1 = b_1' - a_{1n}' x_n - a'_{1\,(n-1)} x_{n-1} - \cdots - a_{12}' x_2$$

Метод Гаусса - пример

$$\begin{cases} 2x_1 + 4x_2 + x_3 = 36 \\ 5x_1 + 2x_2 + x_3 = 47 \\ 2x_1 + 3x_2 + 4x_3 = 37 \\ 5x_1 + x_2 + 4x_3 = 48 \end{cases}$$

$$\begin{cases} x_1 + 2x_2 + \frac{1}{2}x_3 = 18 \\ x_1 + \frac{2}{5}x_2 + \frac{1}{5}x_3 = \frac{47}{5} \\ x_1 + \frac{3}{2}x_2 + 2x_3 = \frac{37}{2} \\ x_1 + \frac{1}{5}x_2 + \frac{4}{5}x_3 = \frac{48}{5} \end{cases}$$

$$\begin{cases} x_1 + 2x_2 + \frac{1}{2}x_3 = 18 \\ -\frac{8}{5}x_2 - \frac{3}{10}x_3 = -\frac{43}{5} \\ -\frac{1}{2}x_2 + \frac{3}{2}x_3 = \frac{1}{2} \\ -\frac{9}{5}x_2 + \frac{3}{10}x_3 = -\frac{42}{5} \end{cases}$$

$$\begin{cases} x_1 + 2x_2 + \frac{1}{2}x_3 = 18 \\ x_2 + \frac{3}{16}x_3 = \frac{43}{8} \\ x_2 - 3x_3 = -1 \\ x_2 - \frac{1}{6}x_3 = \frac{14}{3} \end{cases}$$

$$\begin{cases} x_1 + 2x_2 + \frac{1}{2}x_3 = 18 \\ x_2 + \frac{3}{16}x_3 = \frac{43}{8} \\ -\frac{51}{16}x_3 = -\frac{51}{8} \\ -\frac{17}{48}x_3 = -\frac{17}{24} \end{cases}$$

$$\begin{cases} x_1 + 2x_2 + \frac{1}{2}x_3 = 18 \\ x_2 + \frac{3}{16}x_3 = \frac{43}{8} \\ x_3 = 2 \\ x_3 = 2 \end{cases}$$

$$\begin{cases} x_1 + 2x_2 + \frac{1}{2}x_3 = 18 \\ x_2 + \frac{1}{4}x_3 = \frac{43}{8} \\ x_3 = 2 \\ 0 = 0 \end{cases}$$

$$x_3 = 2$$

$$x_2 = -\frac{3}{16}x_3 + \frac{43}{8} = \frac{43}{8} - \frac{3}{8} = 5$$

$$x_1 = 18 - 2x_2 - \frac{1}{2}x_3 = 18 - 10 - 1 = 7$$

Решение систем линейных уравнений

$$\begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} \cdot \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} = \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix}$$

$$A \cdot X = B$$

Если матрица A невырождена, имеется обратная к ней матрица A^{-1}, и

$$x = A^{-1}b$$

является вектором решения. Можно доказать, что x является единственным решением уравнения, следующим образом. Если имеется два решения, x и x', то Ax = Ax' = b и, обозначая тождественную матрицу как I,

$$x = Ix = (A^{-1}A)x = A^{-1}(Ax) = A^{-1}(Ax') = (A^{-1}A)x' = x'.$$

Рассмотрим решения системы линейных уравнений Ax = b из n уравнений от n неизвестных. Мы можем вычислить A^{-1}, а затем, воспользовавшись уравнением $x = A^{-1}b$, умножить b на A^{-1} и получить $x = A^{-1}b$. На практике этого подхода избегают из-за его численной неустойчивости. К счастью, другой подход - LUP-разложение - численно устойчив и обладает тем преимуществом, что на практике оказывается более быстрым.

Обзор LUP-разложения

Идея, лежащая в основе LUP-разложения, состоит в поиске трех матриц, L, U и P, размером n × n, таких, что

$$PA = LU$$

где

- L - единичная нижнетреугольная матрица;
- U - верхнетреугольная матрица;
- P - матрица перестановки.

Матрицы L, U и P, удовлетворяющие уравнению, называются LUP-разложением (LUP decomposition) матрицы A.

Преимущество вычисления LUP-разложения матрицы A основано на том, что система линейных уравнений решается гораздо легче, если ее матрица треугольна, что и выполняется в случае матриц L и U. Найдя LUP-разложение матрицы A, мы можем решить уравнение Ax = b, путем решения треугольной системы линейных уравнений. Умножая обе части уравнения Ax = b на P, мы получим эквивалентное уравнение PAx = Pb. Используя разложение $PA = LU$, получаем

Обзор LUP-разложения

$$LUx = Pb.$$

Теперь можно решить полученное уравнение, решив две треугольные системы линейных уравнений. Обозначим y = Ux, где x - решение исходной системы линейных уравнений. Сначала решим нижнетреугольную систему линейных уравнений

$$Ly = Pb$$

найдя неизвестный вектор y с помощью метода, который называется прямой подстановкой. После этого, имея вектор y, решим верхнетреугольную систему линейных уравнений

$$Ux = y,$$

найдя x с помощью обратной подстановки. Поскольку матрица перестановки обращаема, умножение обеих частей уравнения на P^{-1} дает $P^{-1}PA = P^{-1}LU$, так что

$$A = P^{-1}LU.$$

Следовательно, вектор является искомым решением системы линейных уравнений Ax = b:

$$Ax = P^{-1}LUx = P^{-1}Ly = P^{-1}Pb = b\ .$$

Прямая и обратная подстановки

Прямая подстановка (forward substitution) позволяет решить нижнетреугольную систему линейных уравнений для данных L, P и за время $\Theta(n^2)$. Для удобства мы представим перестановку P в компактной форме с помощью массива π[1…n]. Элемент π[i] при i = 1, 2, …, n указывает, что $P_{i,\pi[i]} = 1$ и $P_{ij} = 0$ для $j \neq \pi[i]$. Таким образом, в матрице PA на пересечении i-й строки и j-го столбца находится элемент $a_{\pi[i],j}$, а i-м элементом Pb является $b_{\pi[i]}$. Поскольку L является единичной нижнетреугольной матрицей, уравнение можно переписать следующим образом:

$$
\begin{aligned}
y_1 \qquad\qquad\qquad\qquad &= b_{\pi[1]},\\
l_{21}y_1 + \quad y_2 \qquad\qquad &= b_{\pi[2]},\\
l_{31}y_1 + l_{32}y_2 + \quad y_3 \qquad &= b_{\pi[3]},\\
\\
l_{n1}y_1 + l_{n2}y_2 + l_{n3}y_3 + \cdots + y_n &= b_{\pi[n]},
\end{aligned}
$$

Из первого уравнения получаему$_1$ = $b_{\pi[1]}$. Зная значение y_1, его можно подставить во второе уравнение

$$y_2 = b_{\pi[2]} - l_{21}y_1$$

Теперь можно подставить в третье уравнение два найденных значения, y_1 и y_2, и получить

$$y_3 = b_{\pi[3]} - l_{31}y_1 - l_{32}y_2$$

В общем случае для поиска мы подставляем найденные значения y_{i-1} в i-е уравнение и находим

$$y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij}y_j$$

Прямая и обратная подстановки

После того как мы нашли y, найти x можно, воспользовавшись обратной подстановкой (back substitution), которая аналогична прямой. В этом случае мы решаем первым n-е уравнение и работаем в обратном направлении, к первому уравнению. Как и прямая подстановка, этот процесс требует времени $\Theta(n^2)$. Поскольку является верхнетреугольной матрицей, систему линейных уравнений можно переписать как

$$u_{11}x_1 + u_{12}x_2 + \cdots + u_{1,n-2}x_{n-2} + u_{1,n-1}x_{n-1} + u_{1,n}x_n = y_1,$$

$$u_{22}x_2 + \cdots + u_{2,n-2}x_{n-2} + u_{2,n-1}x_{n-1} + u_{2,n}x_n = y_2,$$

$$u_{n-2,n-2}x_{n-2} + u_{n-2,n-1}x_{n-1} + u_{n-2,n}x_n = y_{\mathrm{n}-2},$$

$$u_{n-1,n-1}\,x_{n-1} + u_{n-1,n}x_n = y_{\mathrm{n}-1},$$

$$u_{n,n}x_n = y_{\mathrm{n}}.$$

Таким образом, можно последовательно найти x_n, x_{n-1},…, x_1 следующим образом:

$$x_n = y_{\mathrm{n}}/u_{n,n}\,,$$

$$x_{n-1} = \left(y_{\mathrm{n}-1} - u_{n-1,n}x_n\right)/u_{n-1,n-1}\,,$$

$$x_{n-2} = \left(y_{\mathrm{n}-2} - u_{n-2,n-1}x_{n-1} - u_{n-2,n}x_n\right)/u_{n-2,n-2}\,,$$

или, в общем случае,

$$x_i = (y_{\mathrm{i}} - \sum_{j=i+1}^{n} u_{ij}x_j)/u_{ii}$$

Прямая и обратная подстановки

Процедура LUP-SOLVE для заданных P, L, U и b находит x путем комбинирования прямой и обратной подстановок. В псевдокоде предполагается, что размерность n хранится в атрибуте L.rows и что матрица перестановки P представлена массивом π.

LUP-Solve(L,U,r, b)

1 n = L.rows

2 Пусть x и y - вновь созданные векторы длиной n

3 for i = 1 to n

4 $y_i = b_{\pi[i]} - \sum_{j=1}^{i-1} l_{ij} y_j$

5 for i = n down to l

6 $x_i = (y_i - \sum_{j=i+1}^{n} u_{ij} x_j)/u_{ii}$

7 return x

Процедура LUP-SOLVE находит y в строках 3 и 4 с помощью прямой подстановки, а затем вычисляет x с помощью обратной подстановки в строках 5 и 6.

Наличие внутри каждого цикла for неявного цикла суммирования приводит ко времени работы данной процедуры, равному $\Theta(n^2)$.

Прямая и обратная подстановки - пример

$$\begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{pmatrix} x = \begin{pmatrix} 3 \\ 7 \\ 8 \end{pmatrix},$$

где

$$A = \begin{pmatrix} 1 & 2 & 0 \\ 3 & 4 & 4 \\ 5 & 6 & 3 \end{pmatrix},$$

$$b = \begin{pmatrix} 3 \\ 7 \\ 8 \end{pmatrix},$$

и которую необходимо решить, найдя неизвестный вектор x. LUP-разложение матрицы A имеет вид

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{pmatrix},$$

$$U = \begin{pmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{pmatrix},$$

$$P = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix},$$

Используя прямую подстановку, находим y из Ly = Pb:

$$\begin{pmatrix} 1 & 0 & 0 \\ 0.2 & 1 & 0 \\ 0.6 & 0.5 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 3 \\ 7 \end{pmatrix},$$

получив

$$y = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix},$$

путем вычисления сначала y_1, затем – y_2 и наконец – y_3. Затем с помощью обратной подстановки находим x из Ux = y:

$$\begin{pmatrix} 5 & 6 & 3 \\ 0 & 0.8 & -0.6 \\ 0 & 0 & 2.5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix},$$

тем самым получив искомое решение исходной системы линейных уравнений

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1.4 \\ 1.5 \end{pmatrix}$$

путем вычисления сначала x_3, затем - x_2 и наконец - x_1.

Вычисление LU-разложения

Мы показали, что если можно вычислить LUP-разложение невырожденной матрицы A, то для решения системы линейных уравнений Ax = b можно воспользоваться простыми прямой и обратной подстановками. Осталось показать, как эффективно найти LUP-разложение матрицы A. Начнем со случая невырожденной матрицы A размером n x n и отсутствия матрицы P (или, что эквивалентно, $P = I_n$). В этом случае необходимо найти разложение A = LU. Матрицы L и U называются LU-разложением (LU decomposition) матрицы A.

Итак, мы хотим построить LU-разложение невырожденной матрицы A размером n × n. Если n = 1, задача решена, поскольку мы можем выбрать и U = A. При n > 1 разобьем A на четыре части:

$$A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix}.$$

где $v = (v_2, v_3, \ldots, u_n) = (a_{21}, a_{22}, \ldots, a_{n1})$ представляет собой вектор-столбец длиной (n - 1), $w^T = (w_2, w_3, \ldots, w_n)^T = (a_{12}, a_{13}, \ldots, a_{in})^T$ является вектором-строкой длиной (n - 1), а A' - матрицей размером (n - 1) × (n - 1).

Вычисление LU-разложения

Используя матричную алгебру (проверить полученный результат можно с помощью умножения), разложим матрицу A следующим образом:

$$A = \begin{pmatrix} a_{11} & w^T \\ v & A' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix}$$

Полученная в результате матрица размером (n - 1) × (n - 1)

$$A' - \frac{vw^T}{a_{11}}$$

называется ***дополнением Шура (Schur complement)*** матрицы A по отношению к элементу a_{11}. Мы утверждаем, что если матрица A невырождена, то невырождено и дополнение Шура. Почему? **ДЗ.** Поскольку дополнение Шура невырождено, можно рекурсивно найти его LU-разложение. Запишем

$$A' - vw^T/a_{11} = L'U'$$

где L' - единичная нижнетреугольная матрица, а U' – верхнетреугольная матрица. Тогда, прибегнув к матричной алгебре, получим

$$A = \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & A' - vw^T/a_{11} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{11} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & L'U' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{11} & L' \end{pmatrix} \begin{pmatrix} a_{11} & w^T \\ 0 & U' \end{pmatrix} = LU$$

что дает искомое LU-разложение. Причина, по которой в LUP-разложение включается матрица перестановок P, состоит в том, чтобы избежать деления на нулевые элементы. Использование матрицы перестановки для того, чтобы избежать деления на нуль (или на малые величины, что вносит вклад в численную неустойчивость), называется выбором ведущего элемента (pivoting).

Вычисление LU-разложения

```
LU-Decomposition(A)
n = A.rows
L и U являются новыми матрицами размером n × n
Инициализируем U нулями ниже диагонали
Инициализируем L единицами на диагонали и нулями выше нее
for k = 1 to n
        u_kk = a_kk
    for i = k + 1 to n
        l_ik = a_ik/a_kk        // a_ik хранит v_i
        u_ki = a_ki             // a_ki хранит w_i
    for i = k + 1 to n
        for j = k + 1 to n
            a_ij = a_ij - l_ik l_kj
return L и U
```

Вычисление LU-разложения

Работа процедуры LU-Decomposition. Здесь проиллюстрирована стандартная оптимизация процедуры, при которой мы храним значащие элементы L и U без привлечения дополнительной памяти, непосредственно в матрице A. Иначе говоря, мы можем установить соответствие между каждым элементом a_{ij} и либо l_{ij} (если $i > j$), либо u_{tj} (если $i \le j$) и обновлять матрицу A так, чтобы по окончании работы процедуры в ней хранились матрицы L и U. Чтобы получить псевдокод этой оптимизации из приведенного выше, необходимо просто заменить каждое обращение к l и u на обращение к a; можно легко убедиться в том, что такое преобразование сохраняет корректность алгоритма.

2	3	1	5
6	13	5	19
2	19	10	23
4	10	11	31

2	3	1	5
3	4	2	4
1	16	9	18
2	4	9	21

2	3	1	5
6	**4**	2	4
2	4	1	2
4	1	7	17

2	3	1	5
6	4	2	4
2	4	**1**	2
4	1	7	3

$$\underbrace{\begin{pmatrix} 2 & 3 & 1 & 5 \\ 6 & 13 & 5 & 19 \\ 2 & 19 & 10 & 23 \\ 4 & 10 & 11 & 31 \end{pmatrix}}_{A} = \underbrace{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 3 & 1 & 0 & 0 \\ 1 & 4 & 1 & 0 \\ 2 & 1 & 7 & 1 \end{pmatrix}}_{L} \underbrace{\begin{pmatrix} 2 & 3 & 1 & 5 \\ 0 & 4 & 2 & 4 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 3 \end{pmatrix}}_{U}$$

Вычисление LUP-разложения

В общем случае при решении системы линейных уравнений Ax = b может оказаться, что необходимо выбирать ведущие элементы среди недиагональных элементов матрицы A для того, чтобы избежать деления на 0. Причем нежелательно не только деление на 0, но и просто на малое число, даже если матрица A невырождена, поскольку в результате можно получить численную неустойчивость. В связи с этим в качестве ведущего элемента следует выбирать наибольший возможный элемент.

Математические основы LUP-разложения аналогичны LU-разложению. Вспомним, что имеется невырожденная матрица A размером n x n и требуется найти матрицу перестановки P, единичную нижнетреугольную матрицу L и верхнетреугольную матрицу U, такие, что PA = LU. Перед разделением матрицы A на части, как при вычислении LU-разложения, мы перемещаем ненулевой элемент, скажем, a_{k1}, откуда-то из первого столбца матрицы в позицию (1, 1) (если первый столбец содержит только нулевые значения, то матрица вырождена, поскольку ее определитель равен 0. Для сохранения множества исходных линейных уравнений мы меняем местами строки 1 и k, что эквивалентно умножению матрицы A на матрицу перестановки Q слева.

Вычисление LUP-разложения

Тогда мы можем записать QA как

$$QA = \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix}$$

где v = (a_{21},a_{31},…,a_{n1}), за исключением того, что a_{11} заменяет a_{k1}; w= (a_{k2}, a_{k3}, …, a_{kn})^T; а A является матрицей размером (n - 1) × (n - 1). Поскольку $a_{k1} \neq 0$, можно выполнить почти те же преобразования, что и при LU-разложении, но теперь с гарантией, что деления на нуль не будет:

$$QA = \begin{pmatrix} a_{k1} & w^T \\ v & A' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix}$$

Как мы видели в LU-разложении, если матрица A невырождена, то невырождено и дополнение Шура $A' - vw^T/a_{k1}$. Таким образом, мы можем индуктивно найти его LUP-разложение, дающее единичную нижнетреугольную матрицу L', верхнетреугольную матрицу U' и матрицу перестановки P', такие, что

$$P'(A' - vw^T/a_{k1}) = L'U'$$

Вычисление LUP-разложения

Определим матрицу которая является матрицей перестановки в силу того, что она представляет собой произведение двух матриц перестановки. Мы имеем

$$PA = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} QA = \begin{pmatrix} 1 & 0 \\ 0 & P' \end{pmatrix} \begin{pmatrix} 1 & 0 \\ v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & P' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & A' - vw^T/a_{k1} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & P'(A' - vw^T/a_{k1}) \end{pmatrix}$$

$$= \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & I_{n-1} \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & L'U' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ P'v/a_{k1} & L' \end{pmatrix} \begin{pmatrix} a_{k1} & w^T \\ 0 & U' \end{pmatrix} = LU,$$

что и дает искомое LUP-разложение. Поскольку L' - единичная нижнетреугольная матрица, такой же будет и L, и поскольку U' - верхнетреугольная матрица, такой же будет и U. Заметим, что в отличие от LU-разложения, на матрицу перестановки P' должны умножаться и вектор-столбец v/ak1, и дополнение Шура A' - vw /ak1. Вот как выглядит псевдокод LUP-разложения.

Вычисление LUP-разложения

```
LUP-Decomposition(A)
n = A.rows
π[1…n] - вновь созданный массив
for i = 1 to n
    π[i] = i
for k = 1 to n
    p = 0
    for i = k to n
        if |a_ik| > p
            p = |atk|
            k’ = i
    if p == 0
        error "вырожденная матрица"
    Обменять π[k] с π[k’]
    for i = 1 to n
        Обменять a_ki с a_k’i
    for i = k + 1 to n
        a_ik = a_ik/a_kk
        for j = k + 1 to n
            a_ij = a_ij - a_ik a_kj
```

Вычисление LUP-разложения

Как и в процедуре LUP-Decomposition, в псевдокоде LUP-разложения LUP-Decomposition рекурсия заменяется итерацией. В качестве усовершенствования непосредственной реализации рассмотренной рекурсии мы динамически поддерживаем матрицу перестановки P в виде массива π, где π[i] = j означает, что i-я строка массива P содержит 1 в столбце j. Мы также реализуем код, который вычисляет L и U "на месте", в матрице A, т.е. по окончании работы процедуры

$$a_{ij} = \begin{cases} l_{ij}, & \text{если } i > j \\ u_{ij}, & \text{если } i \leq j \end{cases}$$

В силу тройной вложенности циклов время работы процедуры LUP-Decomposition равно $\Theta(n^3)$, т.е. оно точно такое же, как и в случае процедуры LU- Decomposition. Следовательно, выбор ведущего элемента приводит увеличению времени работы не более чем на постоянный множитель.

Вычисление LUP-разложения

1	2	0	2	0.6
2	3	3	4	-2
3	**5**	5	4	2
4	-1	-2	3.4	-1

3	**5**	5	4	2
2	3	3	4	-2
1	2	0	2	0.6
4	-1	-2	3.4	-1

3	**5**	5	4	2
2	0.6	0	1.6	-3.2
1	0.4	-2	1.4	-2
4	-0.2	-1	4.2	-0.6

3	5	5	4	2
2	0.6	0	1.6	-3.2
1	0.4	**-2**	1.4	-2
4	-0.2	-1	4.2	-0.6

3	5	5	4	2
1	0.4	**-2**	1.4	-2
2	0.6	0	1.6	-3.2
4	-0.2	-1	4.2	-0.6

3	5	5	4	2
1	0.4	**-2**	1.4	-2
2	0.6	0	1.6	-3.2
4	-0.2	0.5	4	-0.5

3	5	5	4	2
1	0.4	-2	1.4	-2
2	0.6	0	1.6	-3.2
4	-0.2	0.5	**4**	-0.5

3	5	5	4	2
1	0.4	-2	1.4	-2
4	-0.2	0.5	**4**	-0.5
2	0.6	0	1.6	-3.2

3	5	5	4	2
1	0.4	-2	1.4	-2
4	-0.2	0.5	**4**	-0.5
2	0.6	0	0.4	-3

$$\underset{P}{\begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}} \underset{A}{\begin{pmatrix} 2 & 0 & 2 & 0.6 \\ 3 & 3 & 4 & -2 \\ 5 & 5 & 4 & 2 \\ -1 & -2 & 3.4 & -1 \end{pmatrix}} = \underset{L}{\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0.4 & 1 & 0 & 0 \\ -0.2 & 0.5 & 1 & 0 \\ 0.6 & 0 & 0.4 & 1 \end{pmatrix}} \underset{U}{\begin{pmatrix} 5 & 5 & 4 & 2 \\ 0 & -2 & 1.4 & -2 \\ 0 & 0 & 4 & -0.5 \\ 0 & 0 & 0 & -3 \end{pmatrix}}$$

Вычисление обратной матрицы из LUP-разложения

Предположим, что имеется LUP-разложение матрицы A на три матрицы, L, U и P, такие, что PA = LU. Используя процедуру LUP-Solve, мы можем решить уравнение вида Ax = b за время $\Theta(n^2)$. Поскольку LUP-разложение зависит только от A, но не от b, мы можем использовать ту же процедуру LUP-SOLVE для решения другой системы линейных уравнений вида Ax = b' за дополнительное время $\Theta(n^2)$. Обобщая, имея LUP-разложение матрицы A, мы можем решить к систем линейных уравнений Ax = b, отличающихся только свободными членами b, за время $\Theta(kn^2)$. Уравнение

$$AX = I_n$$

определяющее матрицу X, обратную A, можно рассматривать как множество из n различных систем линейных уравнений вида Ax = b. Эти уравнения позволяют найти матрицу X, обратную матрице A. Более строго, обозначим i-й столбец Характеристики через X_i и вспомним, что i-м столбцом матрицы I_n является единичный вектор e;. Мы можем найти X в уравнении, использовав LUP-разложение для решения набора уравнений

$$AX_i = e_i$$

для каждого X_i в отдельности. При наличии LUP-разложения поиск каждого столбца X_i требует времени $\Theta(n^2)$, так что полное время вычисления обратной матрицы X на основе LUP-разложения исходной матрицы A требует времени $\Theta(n^3)$.

Умножение матриц и обращение матрицы

Теорема 18.1 (Умножение не сложнее обращения)
Если можно обратить матрицу размером n × n за время I(n), где I(n) = Ω(n^2) и удовлетворяет условию регулярности I(3n) = O(I(n)), то две матрицы размером n × n можно перемножить за время O(I(n)).

Доказательство. Пусть A и B представляют собой матрицы размером n × n, произведение которых C необходимо вычислить. Определим матрицу D размером 3n × 3n как

$$D = \begin{pmatrix} I_n & A & 0 \\ 0 & I_n & B \\ 0 & 0 & I_n \end{pmatrix}.$$

Обратной к матрице D является

$$D^{-1} = \begin{pmatrix} I_n & -A & AB \\ 0 & I_n & -B \\ 0 & 0 & I_n \end{pmatrix},$$

так что мы можем вычислить произведение AB, просто взяв верхнюю правую подматрицу размером n × n из матрицы D^{-1}. Матрицу D можно построить за время Θ(n^2), которое представляет собой O(I(n)), поскольку мы считаем, что I(n) = Ω(n^2), и обратить ее за время O(I(3n)) = O(I(n)) согласно условию регулярности, накладываемому на I(n). Таким образом, M(n) = O(I(n)). Заметим, что I(n) = Θ($n^c \lg^d n$) удовлетворяет условию регулярности при любых константах c > 0 и d > 0.

Умножение матриц и обращение матрицы

Теорема 18.2 (Обращение не сложнее умножения)
Предположим, что мы можем умножить две действительные матрицы размером $n \times n$ за время $M(n)$, где $M(n) = \Omega(n^2)$ и, кроме того, удовлетворяет условиям регулярности $M(n + k) = O(M(n))$ для произвольного $0 \le k \le n$ и $M(n/2) \le cM(n)$ для некоторой константы $c < 1/2$. В таком случае мы можем обратить любую действительную невырожденную матрицу размером $n \times n$ за время $O(M(n))$.

Доказательство. Здесь данная теорема доказывается для действительных чисел.
Можно считать, что n является точной степенью 2, поскольку мы имеем

$$\begin{pmatrix} A & 0 \\ 0 & I_k \end{pmatrix}^{-1} = \begin{pmatrix} A^{-1} & 0 \\ 0 & I_k \end{pmatrix}$$

для любого $k > 0$. Таким образом, выбрав k так, чтобы величина $n + k$ была степенью 2, мы увеличиваем исходную матрицу до размера, представляющего собой степень 2, а искомую обратную матрицу A^{-1} получаем как часть обращенной матрицы большего размера. Первое условие регулярности $M(n)$ гарантирует, что такое увеличение не вызовет увеличения времени работы более чем на постоянный множитель.
Предположим теперь, что матрица A имеет размер n x n и является симметричной и положительно определенной.

Умножение матриц и обращение матрицы

Разобьем матрицу A и обратную к ней A^{-1} на четыре подматрицы размером n/2 × n/2:

$$A = \begin{pmatrix} B & C^T \\ C & D \end{pmatrix} \text{ и } A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix}$$

Обозначив через

$$S = D - CB^{-1}C^T,$$

дополнение Шура матрицы A по отношению к подматрице B, имеем

$$A^{-1} = \begin{pmatrix} R & T \\ U & V \end{pmatrix} = \begin{pmatrix} B^{-1} + B^{-1}C^TS^{-1}CB^{-1} & -B^{-1}C^TS^{-1} \\ -S^{-1}CB^{-1} & S^{-1} \end{pmatrix},$$

поскольку $AA^{-1} = I_n$, как вы можете убедиться, перемножив матрицы. Матрица A симметрична и положительно определена, что и B, и S симметричны и положительно определены. Таким образом, существуют обратные матрицы B^{-1} и S^{-1}, а B^{-1} и S^{-1} симметричны, так что $(B^{-1})^T = B^{-1}$ и $(S^{-1})^T = S^{-1}$. Итак, мы можем вычислить подматрицы R, T, U и V матрицы A^{-1} описанным далее способом.

1. Образуем подматрицы B, C, C^T и D матрицы A.

2. Рекурсивно вычислим обратную матрице B матрицу B^{-1}.

3. Вычислим произведение матриц $W = CB^{-1}$, а затем транспонируем его W^T, получив матрицу, эквивалентную $B^{-1}C^T$ ($(B^{-1})^T = B^{-1}$).

4. Вычислим произведение матриц $X = WC^T$, эквивалентное $CB^{-1}C^T$, затем - матрицу $S = D - X = D - CB^{-1}C^T$.

5. Рекурсивно вычислим обратную матрицу S-1 и присвоим ее матрице V.

6. Вычислим произведение матриц $Y = S^{-1}W$, равное $S^{-1}CB^{-1}$, а затем транспонируем его Y^T, что равно $B^{-1}C^TS^{-1}$ ($(B^{-1})^T= B^{-1}$ и $(S^{-1})^T = S^{-1}$). Присвоим T значение $-Y^T$, а U - значение -Y.

7. Вычислим произведение матриц $Z = W^TY$, равное $B^{-1}C^TS^{-1}CB^{-1}$, и присвоим R значение $B^{-1} + Z$.

Умножение матриц и обращение матрицы

Таким образом, можно инвертировать симметричную положительно определенную матрицу размером n × n путем обращения двух матриц размером n/2 × n/2, последующего выполнения четырех перемножений матриц размером n/2 × n/2, а также выполнения дополнительных действий по извлечению подматриц из A, вставке подматриц в A^{-1} ценой $O(n^2)$, и константного количества сложений, вычитаний и транспонирований матриц размером n/2×n/2. В результате мы получаем следующее рекуррентное соотношение:

$$I(n) \leq 2I\left(\frac{n}{2}\right) + 4M\left(\frac{n}{2}\right) + O(n^2) = 2I(n/2) + \Theta(M(n)) = O(M(n)).$$

Вторая строка выполняется, поскольку из второго условия регулярности в формулировке теоремы вытекает, что $4M(n/2) < 2M(n)$, и поскольку мы предполагаем, что $M(n) = \Omega(n^2)$.

Остается доказать, что асимптотическое время умножения матриц может быть получено для обращения невырожденной матрицы A, которая не является симметричной и положительно определенной. Основная идея заключается в том, что для любой невырожденной матрицы A матрица A^TA и положительно определенная. Все, что остается, — это привести задачу обращения матрицы A к задаче обращения матрицы A^TA.

Такое приведение основано на наблюдении, что если A является невырожденной матрицей размером n × n, то $A^{-1} = (A^TA)^{-1}A^T$, поскольку $((A^TA)^{-1}A^T)A = (A^TA)^{-1}(A^TA) = I_n$, а обратная матрица единственна. Следовательно, можно вычислить A^{-1}, сначала умножив A^T на A для получения симметричной положительно определенной матрицы A^TA, а затем обратив эту матрицу с помощью описанного выше рекурсивного алгоритма и умножив полученный результат на A^T. Каждый из перечисленных шагов требует времени O(M(n)), так что обращение любой невырожденной матрицы с действительными элементами может быть выполнено за время O(M(n)).

Симметричные положительно определенные матрицы и метод наименьших квадратов

Лемма 18.1 Любая симметричная положительно определенная матрица является невырожденной.

Доказательство. Предположим, что матрица A вырождена. Тогда имеется такой ненулевой вектор x, что Ax = 0. Следовательно, $x^TAx = 0$, и A не может быть положительно определенной.

Доказательство того факта, что LU-разложение симметричных положительно определенных матриц можно выполнить, не опасаясь столкнуться с делением на 0, более сложное. Начнем с доказательства свойств некоторых определенных подматриц A. Определим k-ю ***главную подматрицу*** (leading submatrix) A как матрицу A_k, состоящую из пересечения к первых строк и k первых столбцов A.

Симметричные положительно определенные матрицы и метод наименьших квадратов

Лемма 18.2 Если A представляет собой симметричную положительно определенную матрицу, то все ее главные подматрицы симметричные и положительно определенные.

Доказательство. То, что каждая главная подматрица A_k является симметричной, очевидно. Для доказательства того, что она положительно определенная, воспользуемся методом от противного. Если не является положительно определенной, то существует вектор $x_k \neq 0$ размером k, такой, что $x_k^T A_k x_k$. Пусть матрица A имеет размер n × n и

$$A = \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix}$$

с подматрицами B (размером (n - k) × k) и C (размером (n - k) × (n - k)). Определим вектор $x = (x_k^T\ 0)^T$ размером n, в котором после x следуют n – k нулей. Тогда

$$x_k^T A x = \begin{pmatrix} x_k^T & 0 \end{pmatrix} \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} x_k \\ 0 \end{pmatrix} = \begin{pmatrix} x_k^T & 0 \end{pmatrix} \begin{pmatrix} A_k x_k \\ B x_k \end{pmatrix} = x_k^T A_k x_k \leq 0$$

что противоречит условию, что матрица A положительно определенная.

Симметричные положительно определенные матрицы и метод наименьших квадратов

Лемма 18.3 (Лемма о дополнении Шура) Если A представляет собой симметричную положительно определенную матрицу, а A_k - главная подматрица A размером k, то дополнение Шура S матрицы A относительно подматрицы Ак является симметричным положительно определенным.

Доказательство. Поскольку матрица A симметрична, симметрична также подматрица C. Дополнение Шура S также является симметричным.

Остается показать, что дополнение Шура S положительно определенное. Для любого ненулевого вектора x в соответствии с предположением о том, что A является положительно определенной матрицей, выполняется соотношение $x^TAx > 0$. Разобьем x на два подвектора, y и z, совместимые с Ак и C соответственно. В силу существования A имеем

$$x^TAx = \begin{pmatrix} y^T & z^T \end{pmatrix} \begin{pmatrix} A_k & B^T \\ B & C \end{pmatrix} \begin{pmatrix} y \\ z \end{pmatrix} = \begin{pmatrix} y^T & z^T \end{pmatrix} \begin{pmatrix} A_k y + B^T z \\ By + Cz \end{pmatrix}$$

$$= y^TA_ky + y^TB^Tz + z^TBy + z^TCz$$

$$= \left(y + A_k^{-1}B^Tz\right)^T A_k \left(y + A_k^{-1}B^Tz\right) + z^T\left(C - BA_k^{-1}B^T\right)z.$$

Поскольку неравенство $x^TAx > 0$; $z^T\left(C - BA_k^{-1}B^T\right)z = z^TSz = x^TAx > 0.$

Метод наименьших квадратов

Одним из важных приложений симметричных положительно определенных матриц является подбор кривой для заданного множества экспериментальных точек. Предположим, что дано множество из m точек
(x_1,y_1), (x_2, y_2),… ,(x_m,y_m),
где значения y, содержат ошибки измерений. Нужно найти функцию F(x), такую, что ошибки аппроксимации

$$\eta_i = F(x_i) - y_i$$

малы при i = 1,2,…,m. Вид функции F зависит от рассматриваемой задачи, и здесь мы будем считать, что она имеет вид линейной взвешенной суммы

$$F(x) = \sum_{j=1}^{n} c_j f_j(x),$$

где количество слагаемых n и набор базисных функций (basis functions) f выбираются на основе знаний о рассматриваемой задаче. Зачастую в качестве базисных функций выбираются $f_j(x) = x^{j-1}$, т.е. функция F представляет собой полином степени n - 1 от x:

$$F(x) = c_1 + c_2 x + c_3 x^2 + \cdots + c_n x^{n-1}.$$

Таким образом, для заданных n экспериментальных точек (x_1,y_1), (x_2, y_2),… ,(x_m,y_m) необходимо вычислить n коэффициентов c_1, c_2,…, c_n, минимизирующих ошибки приближения $\eta_1, \eta_2, \dots, \eta_m$.
Выбрав n = m, можно точно вычислить все y_i.

Метод наименьших квадратов

Выбор такой функции F с высокой степенью не слишком удачен, так как, помимо данных, он учитывает и весь "шум", что приводит к плохим результатам при использовании F для предсказания значения y для некоторого x, измерения для которого еще не выполнялись. В любом случае, когда выбрано n, меньшее, чем m, мы получаем переопределенную систему линейных уравнений, приближенное решение которой хотим найти. Пусть

$$A = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \cdots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \cdots & f_n(x_m) \end{pmatrix}$$

обозначает матрицу значений базисных функций в заданных точках, т.е. $a_{ij} = f_i(x_j)$, и пусть c = (c_k) обозначает искомый вектор коэффициентов размером n.

Тогда

$$Ac = \begin{pmatrix} f_1(x_1) & f_2(x_1) & \cdots & f_n(x_1) \\ f_1(x_2) & f_2(x_2) & \cdots & f_n(x_2) \\ \vdots & \vdots & \ddots & \vdots \\ f_1(x_m) & f_2(x_m) & \cdots & f_n(x_m) \end{pmatrix} \begin{pmatrix} c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} F(x_1) \\ F(x_2) \\ \vdots \\ F(x_n) \end{pmatrix}$$

представляет собой вектор размером m "предсказанных значений" y, а вектор

$$\eta = Ac - y$$

является вектором невязок (ошибок приближения - approximation error) размером m.

Метод наименьших квадратов

Для минимизации невязок будем минимизировать норму вектора ошибок η, что отражено в названии "решение методом наименьших квадратов" (least-squa resolution), так как

$$||\eta|| = \left(\sum_{i=1}^{m} \eta_i^2\right)^{1/2}$$

Поскольку

$$||\eta||^2 = ||Ac - y||^2 = \sum_{i=1}^{m}\left(\sum_{j=1}^{n} a_{ij}c_j - y_i\right)^2,$$

можно минимизировать $||\eta||^2$, дифференцировав $||\eta||^2$ по всем c_k и приравняв полученные производные к 0:

$$\frac{d||\eta||^2}{dc_k} = \sum_{i=1}^{m} 2\left(\sum_{j=1}^{n} a_{ij}c_j - y_i\right)a_{ik} = 0.$$

n уравнений с k = 1, 2, …, n эквивалентны одному матричному уравнению
$(Ac - y)^T A = 0$,
или, что то же самое,
$A^T(Ac - y) = 0$,
откуда вытекает

$$A^T Ac = A^T y.$$

Метод наименьших квадратов

В качестве примера рассмотрим пять экспериментальных точек,
$(x_1,y_1) = (-1,2)$, $(x_2,y_2) = (1,1)$, $(x_3, y_3) = (2,1)$, $(x_4,y_4) = (3,0)$, $(x_5, y_5) = (5,3)$,
Необходимо найти приближение экспериментальных данных квадратичным полиномом
$F(x) = c_1 + c_2x + c_3x^2$.
Начнем с матрицы значений базисных функций:

$$A = \begin{pmatrix} 1 & x_1 & x_1^2 \\ 1 & x_2 & x_2^2 \\ 1 & x_3 & x_3^2 \\ 1 & x_4 & x_4^2 \\ 1 & x_5 & x_1^2 \end{pmatrix} = \begin{pmatrix} 1 & -1 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \\ 1 & 3 & 9 \\ 1 & 5 & 25 \end{pmatrix}.$$

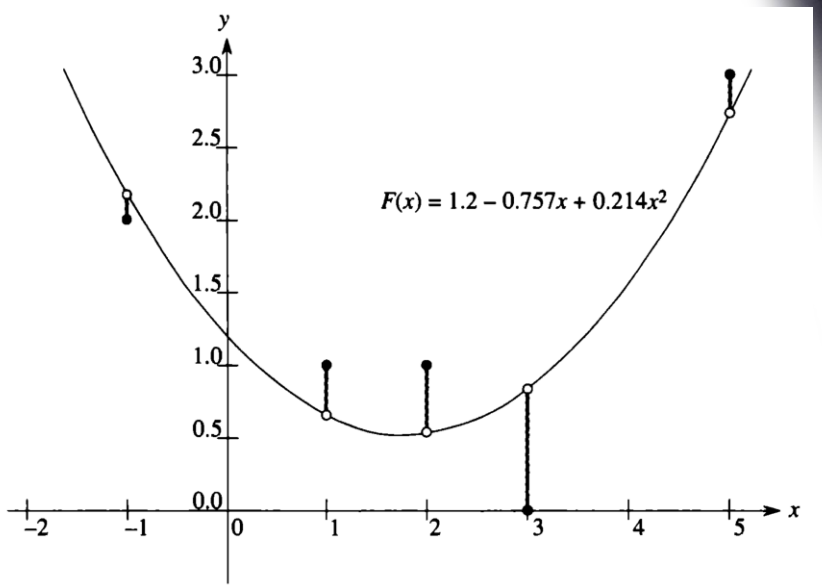
Ее псевдообратная матрица имеет вид

$$A^+ = \begin{pmatrix} 0.500 & 0.300 & 0.200 & 0.100 & -0.100 \\ -0.388 & 0.093 & 0.190 & 0.193 & -0.088 \\ 0.060 & -0.036 & -0.048 & -0.036 & 0.060 \end{pmatrix}$$

Умножив y на A^+, получаем вектор коэффициентов

$$c = \begin{pmatrix} 1.200 \\ -0.757 \\ 0.214 \end{pmatrix}$$

$F(x) = 1.200 - 0.757x + 0.214x^2$, который представляет собой наилучшее квадратичное приближение экспериментальных данных.



Итерационные методы

Сложность $\mathcal{O}(n^2 t)$, t – число шагов.

Пусть дана система линейных алгебраических уравнений:

Ax = F

Выбирается начальное приближение $x^{(0)} = \left(x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)}\right)$ (при отсутствии априорных данных для выбора приближения в качестве начального приближения можно выбрать нулевой вектор).

<u>Метод Якоби</u>. Каждое следующее приближение в методе Якоби рассчитывается по формуле:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{1}{a_{ii}}\left[f_i - \sum_{j=1}^{n} a_{ij} x_j^{(k)}\right], i = 1, \dots, n$$

где *k* – номер текущей итерации.

Метод Гаусса-Зейделя. Каждое последующее приближение рассчитывается по формуле:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{1}{a_{ii}}\left[f_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij} x_j^{(k)}\right], i = 1, \dots, n$$

Для ускорения сходимости итерационного процесса можно использовать *параметр релаксации*.

Итерационные методы

Итерационный процесс в ***методе Якоби с параметром релаксации*** выглядит следующим образом:

$$\hat{x}_i^{(k+1)} = x_i^{(k)} + \frac{1}{a_{ii}}\left[f_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij}x_j^{(k)}\right],$$

$$x_i^{(k+1)} = w\hat{x}_i^{(k+1)} + (1-w)x_i^{(k)}$$

Подставляя в приближение в методе Якоби, получаем:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{w}{a_{ii}}\left[f_i - \sum_{j=1}^{n} a_{ij}x_j^{(k)}\right], 0 < w \leq 1$$

В методе ***Гаусса-Зейделя с параметром релаксации*** (другое название - ***метод релаксации***) итерационный процесс описывается следующим образом:

$$x_i^{(k+1)} = x_i^{(k)} + \frac{w}{a_{ii}}\left[f_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i}^{n} a_{ij}x_j^{(k)}\right], 0 < w < 2$$

Условия выхода из итерационного процесса для рассмотренных методов:

1. Выход по относительной невязке: $\frac{\|F - Ax^{(k)}\|}{\|F\|} < \varepsilon$
2. Защита от зацикливания: *k* > *maxiter*, *maxiter* - максимальное количество итераций.

Решение систем нелинейных уравнений методом Ньютона

Пусть дана СНУ в виде:

$$F_1(x_1, x_2, \dots, x_n) = 0;$$
$$F_2(x_1, x_2, \dots, x_n) = 0;$$
$$\vdots$$
$$F_m(x_1, x_2, \dots, x_n) = 0;$$

Обозначим через x^k решение, полученное на k-й итерации процесса Ньютона (для первой итерации x^0 – начальное приближение). Запишем исходную систему в виде $F_1\left(x^k + \Delta x^k\right) = 0\,, i\ = 1 \dots m,$ где $\Delta x^k = \bar{x}\ -\ x^k\,, \bar{x}$ – искомое решение.

Выполним линеаризацию i-го уравнения системы с использованием его разложения в ряд Тейлора в окрестности точки x^k :

$$F_i(x) \approx F_i\left(x^k\right) + \sum_{j=1}^{n} \left.\frac{\partial F_i(x)}{\partial x_j}\right|_{x=x^k} \Delta x_j^k, i = 1 \dots m$$

или, в матричном виде:

$$A^k\,\Delta x^k = -F^k,$$

где F^k – значение вектор-функции F при x = x^k; A^k – матрица Якоби ($A_{ij}^k = \left.\frac{\partial F_i(x)}{\partial x_j}\right|_{x=x^k}$).

Решение систем нелинейных уравнений методом Ньютона

Это система уравнений, линейных относительно приращений Δx_j^k. Решив эту систему, найдем направление Δx^k поиска решения.

Для поиска следующего приближения x^{k+1} вдоль направления Δx^k организуем итерационный процесс:

$$x_v^{k+1} = x^k + \beta_v^k \Delta x^k,$$

где β_v^k – параметр итерационного процесса поиска x^{k+1}, (0 < β^k <1), v – номер итерации поиска оптимального значения β^k. Параметр β^k будем искать следующим образом: сначала (то есть после нахождения направления Δx^k) β^k принимается равным 1 и вычисляется значение $F_v^k = F(x^k + \beta_v^k \Delta x^k)\, n$; далее, пока норма F_v^k больше, чем норма F^{k-1}, β_v^k уменьшается вдвое.

Заметим, что в СЛАУ матрица A^k при несовпадении числа неизвестных и числа уравнений становится прямоугольной. В этом случае вместо СЛАУ решают другую (измененную) СЛАУ с квадратной матрицей, решение которой является решением.

План лекции

Командная строка	Решение конечных уравнений	Решение СЛАУ	Вычислительная геометрия
10 минут	20 минут	40 минут	20 минут

Основы

Вычислительная геометрия - это раздел информатики, изучающий алгоритмы, предназначенные для решения геометрических задач. В современных инженерных и математических расчетах вычислительная геометрия, в числе других областей знаний, применяется в машинной графике, в робототехнике, при разработке СБИС, при автоматизированном проектировании, в металлургии, в статистике…

Выпуклой комбинацией (convex combination) двух различных точек $p_1 = (x_1,y_1)$ и $p_2 = (x_2,y_2)$ называется любая точка $p_3 = (x_3, y_3)$, такая, что для некоторого значения a, принадлежащего интервалу $0 < \alpha < 1$, выполняются равенства

$x_3 = x_1 + (1 - \alpha)x_2$ и

$y_3 = \alpha y_1 + (1 - a)y_2$ (пишут также $p_3 = \alpha p_1 + (1 - \alpha)p_2$).

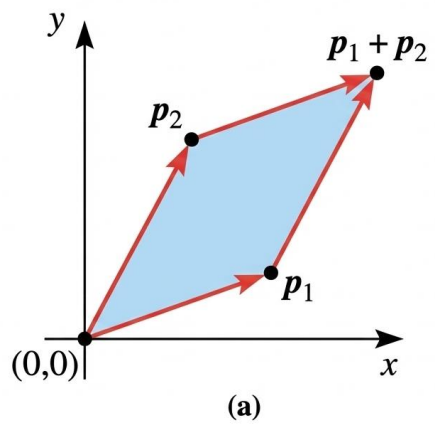
В роли p_3 может выступать любая точка, которая принадлежит прямой, соединяющей точки p_1 и p_2, и находится между этими точками.

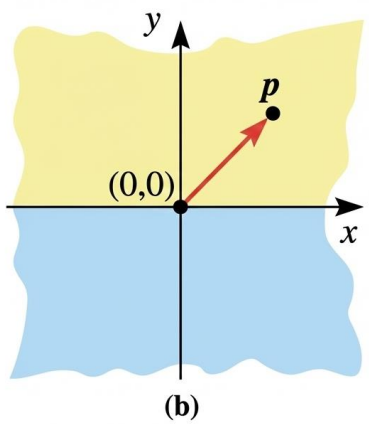
Если заданы две различные точки, p_1 и p_2, то ***отрезком*** (line segment) $\overline{p_1p_2}$ называется ***множество выпуклых комбинаций*** p_1 и p_2. Точки p_1 и p_2 называются ***конечными точками*** (endpoints) отрезка $\overline{p_1p_2}$. Иногда играет роль порядок следования точек p_1 и p_2, и тогда говорят о ***направленном отрезке*** (directed segment) $\overrightarrow{p_1p_2}$. Если точка p_1 совпадает с ***началом координат*** (origin), т.е. имеет координаты (0,0), то направленный отрезок $\overrightarrow{p_1p_2}$ можно рассматривать как ***вектор*** (vector) p_2.

Векторное произведение

Вычисление векторного произведения составляет основу методов работы с отрезками. Рассмотрим векторы p_1 и p_2. ***Векторное произведение*** (cross product) $p_1 \times p_2$ можно интерпретировать как знаковое значение площади параллелограмма, образованного точками (0,0), p_1, p_2 и $p_1 + p_2 = (x_1 + x_2, y_1 + y_2)$. Эквивалентное, но более полезное определение векторного произведения - определитель матрицы:

$$p_1 \times p_2 = det\begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} = x_1 y_2 - x_2 y_1 = -p_2 \times p_1.$$





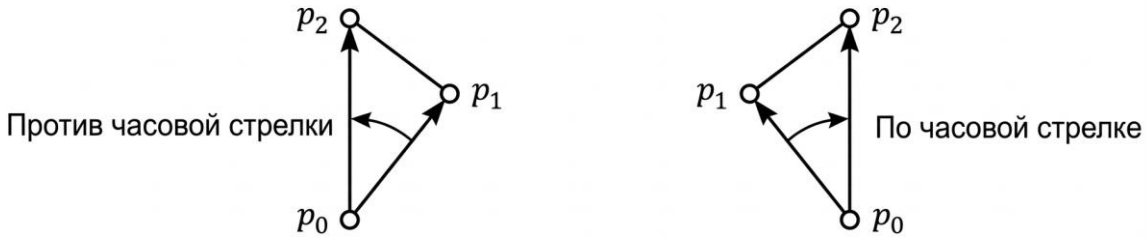
Чтобы определить, находится ли направленный отрезок $\overrightarrow{p_0 p_1}$ по часовой стрелке от направленного отрезка $\overrightarrow{p_0 p_2}$ или против относительно их общей точки p_0, достаточно использовать ее как начало координат. Сначала обозначим величину $p_1 - p_0$ как вектор $p'_1 = (x_1', y_1')$, где $x_1' = x_1 - x_0$ и $y_1' = y_1 - y_0$, а затем введем аналогичные обозначения для величины $p_2 - p_0$. После этого вычислим векторное произведение

$$(p_1 - p_0) \times (p_2 - p_0) = (x_1 - x_0)(x_2 - x_0) - (x_2 - x_0)(y_1 - y_0).$$

Если это векторное произведение положительно, $\overrightarrow{p_0 p_1}$ находится по часовой стрелке от $\overrightarrow{p_0 p_2}$; если отрицательно - против часовой стрелки.

Поворот последовательных отрезков

Следующий вопрос заключается в том, куда сворачивают два последовательных отрезка, $\overline{p_0p_1}$ и $\overline{p_1p_2}$, в точке p_1 - влево или вправо. Можно привести эквивалентную формулировку этого вопроса - определить знак угла $\angle\overrightarrow{p_1p_2}$,. Векторное произведение позволяет ответить на этот вопрос, не вычисляя величину угла.



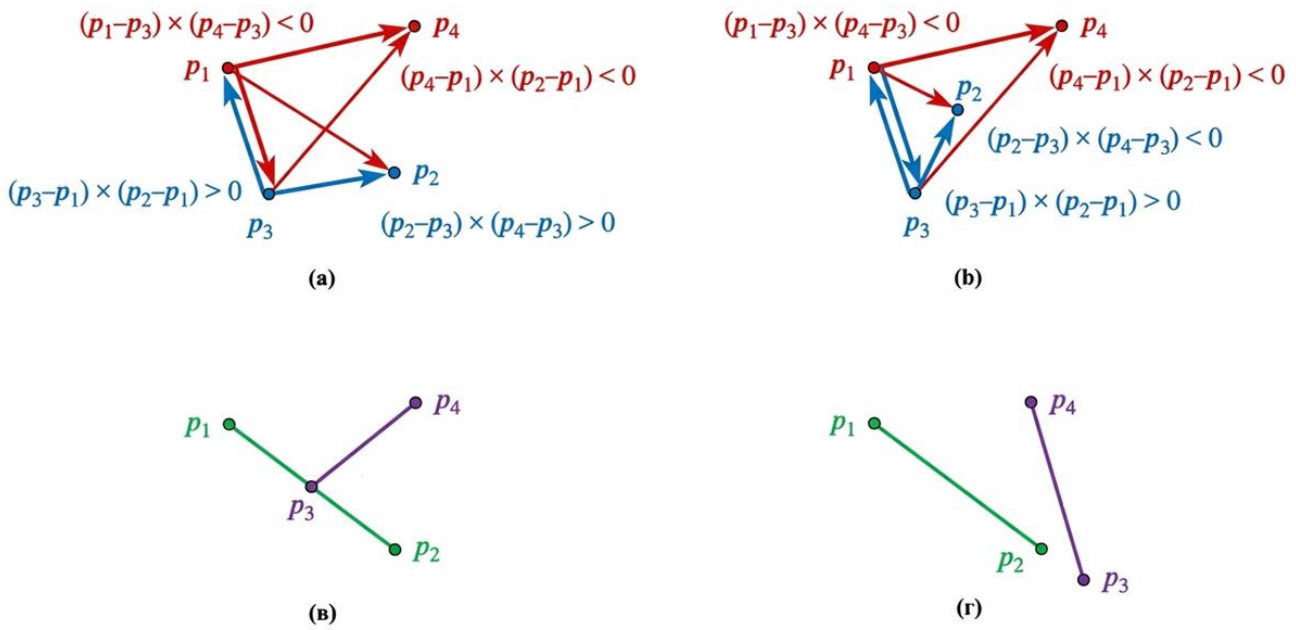
Мы просто проверяем, находится ли направленный отрезок $\overrightarrow{p_0p_2}$ по часовой стрелке или против часовой стрелки относительно направленного отрезка $\overrightarrow{p_0p_1}$. Для этого вычисляется векторное произведение $(p_2 - p_0) \times (p_1 - p_0)$.

Если эта величина отрицательна, то направленный отрезок $\overrightarrow{p_0p_2}$ находится против часовой стрелки по отношению к направленному отрезку $\overrightarrow{p_0p_1}$, так что в точке p_1 мы делаем поворот влево. Положительное значение векторного произведения указывает на ориентацию по часовой стрелке и поворот вправо. Нулевое векторное произведение означает, что точки p_0, p_1 и p_2 коллинеарны.

Пересекаются ли два отрезка?

Чтобы определить, пересекаются ли два отрезка, следует проверить, пресекает ли каждый из них прямую, содержащую другой отрезок. Отрезок $\overline{p_1 p_2}$ пересекает (straddles) прямую, если конечные точки отрезка p_1 и p_2 лежат в разных полуплоскостях, на которые прямая разбивает плоскость. В граничном случае точка p_1 или точка p_2 (или обе эти точки) лежит непосредственно на прямой. Два отрезка пересекаются тогда и только тогда, когда выполняется одно из сформулированных ниже условий (или оба эти условия одновременно).

1. Каждый отрезок пересекает прямую, на которой лежит другой отрезок.
2. Конечная точка одного из отрезков лежит на другом отрезке (граничный случай).



Пересекаются ли два отрезка?

```
#include <stdio.h>
#include <stdbool.h>
// Структура для точки
typedef struct {
    int x, y;
} Point;
// Функция направления (векторное произведение)
int Direction(Point pi, Point pj, Point pk) {
    return ((pk.x - pi.x) * (pj.y - pi.y)) - ((pk.y - pi.y) * (pj.x - pi.x));
}
// Проверка, лежит ли точка pk на отрезке pi-pj
bool OnSegment(Point pi, Point pj, Point pk)
{
    int min_x = pi.x < pj.x ? pi.x : pj.x;
    int max_x = pi.x > pj.x ? pi.x : pj.x;
    int min_y = pi.y < pj.y ? pi.y : pj.y;
    int max_y = pi.y > pj.y ? pj.y : pi.y;
    return (min_x <= pk.x && pk.x <= max_x) && (min_y <= pk.y && pk.y <= max_y);
}
```

Пересекаются ли два отрезка?

```
// Проверка пересечения отрезков p1-p2 и p3-p4
bool SegmentsIntersect(Point p1, Point p2, Point p3, Point p4) {
    int d1 = Direction(p3, p4, p1);
    int d2 = Direction(p3, p4, p2);
    int d3 = Direction(p1, p2, p3);
    int d4 = Direction(p1, p2, p4);
    // Основной случай: отрезки пересекаются, если направления противоположны
    if (((d1 > 0 && d2 < 0)||(d1 < 0 && d2 > 0))&&((d3 > 0 && d4 < 0)||(d3 < 0 && d4 > 0)))
        return true;
    // Проверка особых случаев (точка на отрезке)
    else if (d1 == 0 && OnSegment(p3, p4, p1))
        return true;
    else if (d2 == 0 && OnSegment(p3, p4, p2))
        return true;
    else if (d3 == 0 && OnSegment(p1, p2, p3))
        return true;
    else if (d4 == 0 && OnSegment(p1, p2, p4))
        return true;
    else
        return false;
}
```

Определение наличия пересекающихся отрезков

В этом алгоритме используется метод, известный под названием ***"выметание"***, который часто встречается в алгоритмах вычислительной геометрии. С помощью данного алгоритма или его вариации можно решать и другие задачи вычислительной геометрии. Время работы этого алгоритма равно O(nlgn), где n - количество заданных отрезков. В нем лишь определяется, существуют пересечения или нет, но не выводятся данные обо всех этих пересечениях.

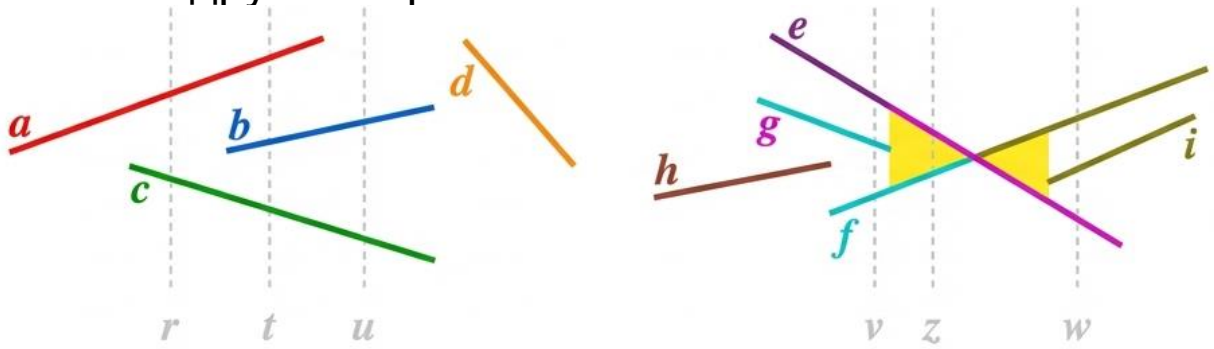
В методе ***выметания*** (sweeping) по заданному множеству геометрических объектов проводится воображаемая вертикальная ***выметающая прямая*** (sweep line), которая обычно движется слева направо. Измерение, вдоль которого двигается выметающая прямая (в данном случае это измерение x), трактуется как время. Выметание предоставляет способ упорядочения геометрических объектов, обычно путем размещения их параметров в динамической структуре данных, что позволяет воспользоваться взаимоотношениями между этими объектами.

Чтобы описать алгоритм и доказать его способность корректно определить, пересекаются ли какие-либо из n отрезков, сделаем два упрощающих предположения.

Упорядочение отрезков

Поскольку предполагается, что вертикальные отрезки отсутствуют, каждый входной отрезок пересекает данную вертикальную выметающую прямую в одной точке. Таким образом, отрезки, пересекающие вертикальную выметающую прямую, можно упорядочить по координате y точки пересечения. Чтобы быть более точными, рассмотрим два отрезка, s_1 и s_2. Говорят, что они ***сравнимы*** (comparable) в координате x, если вертикальная выметающая прямая с координатой x пересекает оба этих отрезка. Говорят также, что отрезок s_1 ***расположен над*** (above) отрезком s_2 в x (записывается $s_1 \succcurlyeq_x s_2$), если отрезки s_1 и s_2 сравнимы в координате x и точка пересечения отрезка s_1 с выметающей прямой в координате x находится выше, чем точка пересечения отрезка s_2 с этой выметающей прямой, или если s_1 и s_2 пересекаются на выметающей прямой.

Пример, выполняются соотношения $a \succcurlyeq_r c$, $a \succcurlyeq_t b$, $b \succcurlyeq_t c$, $a \succcurlyeq_t c$ и $b \succcurlyeq_u c$. Отрезок d не сравним ни с каким другим отрезком.



(а) (б)

Перемещение выметающей прямой

В выметающих алгоритмах обычно обрабатываются два набора данных.

1. ***Состояние относительно выметающей прямой*** (sweep-line status) описывает соотношения между объектами, пересекаемыми выметающей прямой.

2. ***Таблица, или расписание, точек-событий*** (event-point schedule) – это последовательность координат x, упорядоченных слева направо, в которой определяются точки останова выметающей прямой. Каждая такая точка останова называется точкой-событием (event point). По мере перемещения выметающей прямой слева направо, когда она достигает x-координаты точки-события, выметание приостанавливается, выполняется обработка точки-события, после чего выметание продолжается. Изменение состояния относительно выметающей прямой происходит только в точках-событиях.

Состояние относительно выметающей прямой является полностью упорядоченным множеством T, в котором необходимо реализовать такие операции.

Insert(T, s): вставка отрезка s в T.

Delete(T, s): удаление отрезка s из T.

Above(T, s): возврат отрезка, находящегося непосредственно над отрезком s в T.

Below(T, s): возврат отрезка, находящегося непосредственно под отрезком s в T.

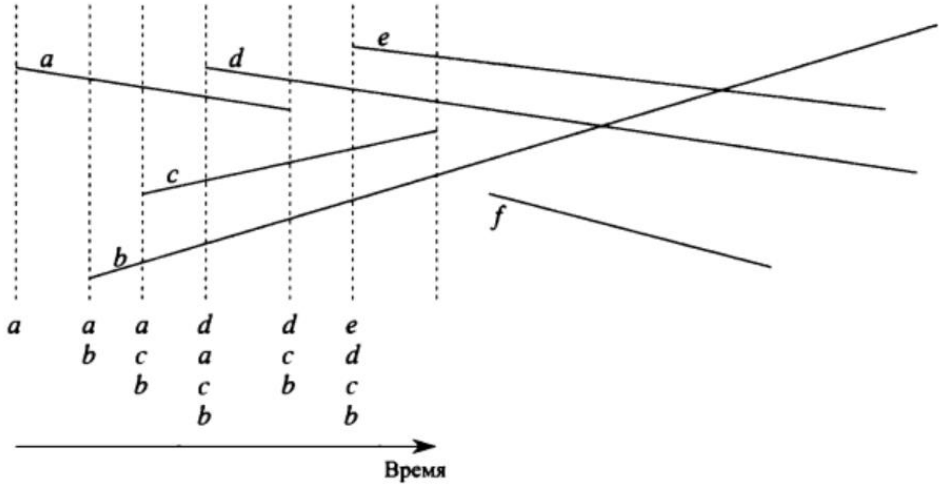
Возможна ситуация, когда отрезки s_1 и s_2 находятся взаимно один над другим в упорядочении T; такая ситуация может возникнуть, если s_1 и s_2 пересекаются на выметающей прямой, для которой задается упорядочение T. В этом случае эти два отрезка могут находиться в T в любом порядке.

Если всего имеется n входных отрезков, каждую из перечисленных выше операций с помощью красно-черных деревьев можно выполнить за время O(lgn).

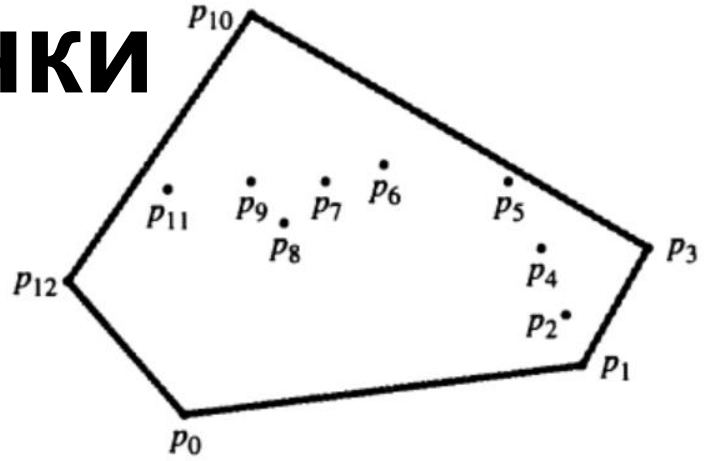
Перемещение выметающей прямой

Any-Segments-Intersect(S)
1 T = Ø
2 Сортировка конечных точек отрезков S слева направо с разрешением совпадений путем помещения левых конечных точек перед правыми и путем помещения точек с меньшими координатами y перед теми, координата y которых больше
3 for каждой точки p в отсортированном списке конечных точек
4 if p является левой конечной точкой отрезка s
5 Insert(T, s)
6 if (Above(T, s) существует и пересекает s) или (Below(T, s) существует и пересекает s)
7 return TRUE
8 if p является правой конечной точкой отрезка s
9 if (и Above(T, s), и Below(T, s) существуют) и (Above(T,s) пересекает Below(T,s))
10 return TRUE
11 Delete(T,s)
12 return FALSE

Строка 2 - слиянием или пирамидальная сортировки **O(nlogn)**. Цикл for в строках 3-11 выполняет не более одной итерации на точку, **2n** раз. На каждую итерацию требуется время O(logn), поскольку выполнение каждой операции в красно-черном дереве занимает время O(logn), и каждая проверка пересечения выполняется за время O(1). Таким образом, полное время работы равно O(nlgn).



Поиск выпуклой оболочки



Выпуклой оболочкой (convex hull) множества точек Q (обозначается CH(Q)) называется наименьший выпуклый многоугольник P, такой, что каждая точка из Q находится либо на границе многоугольника P, либо в его внутренней области. Предполагается, что все точки Q различны и что в нем имеется как минимум три не коллинеарные точки.

В **инкрементном методе** (incremental method) точки сначала сортируются слева направо, в результате чего получается последовательность (p_1, p_2, p_n). На i-м этапе выпуклая оболочка i - 1 крайних слева точек CH({p_1,p_2, p_{i-1}}) обновляется в соответствии с положением i-й слева точки, формируя, таким образом, оболочку CH({p_1,p_2,p_i}).

В **методе декомпозиции** (devide-and-conquer method) множество n точек за время Ѳ(n) разбивается на два подмножества, в одном из которых содержится [n/2] крайних слева точек, а во втором - [n/2] крайних справа. Затем рекурсивно вычисляются выпуклые оболочки этих подмножеств, которые впоследствии объединяются с помощью одного остроумного метода за время O(n). Время работы T(n) = 2T(n/2) + O(n), решение которого дает время работы **O(nlgn)**.

Метод отсечения и поиска (prune-and-search method) - время работы в наихудшем случае ведет себя линейно. В нем строится верхняя часть (или "верхняя цепь") выпуклой оболочки путем многократного отбрасывания фиксированной части оставшихся точек до тех пор, пока не останется только верхняя цепь выпуклой оболочки. Затем то же самое выполняется с нижней цепью. Если выпуклая оболочка содержит h вершин, время его работы O(nlgh).

Сканирование по Грэхему

Graham-Scan(Q)

1 Пусть p_0 - точка Q с минимальной координатой y, или крайняя слева из таких точек при наличии совпадений

2 Пусть (p_1, p_2, p_m) - остальные точки Q, отсортированные в порядке возрастания полярного угла, измеряемого против часовой стрелки относительно p_0 (если полярные углы нескольких точек совпадают, то из множества удаляются все эти точки, кроме одной, самой дальней от точки p_0)

3 if m < 2

4 return "выпуклая оболочка пуста"

5 else пусть S - пустой стек

6 Push(p_0, S)

7 Push(p_1, S)

8 Push(p_2, S)

9 for i = 3 to m

10 while угол, образованный точками Next-To-Top(S),
Top(S) и p, не образует поворот влево

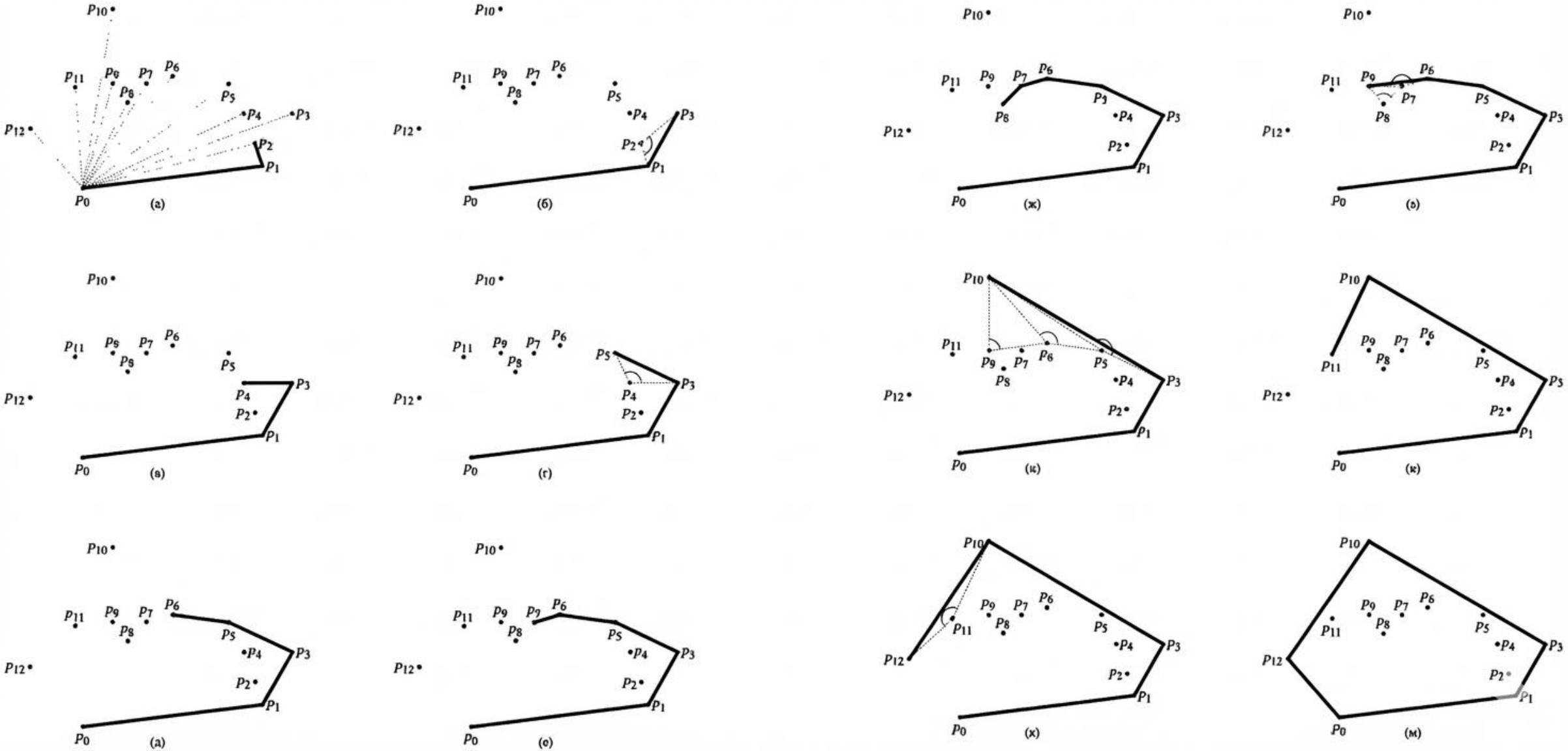
11 Pop(S)

12 Push(p_1, S)

13 return S

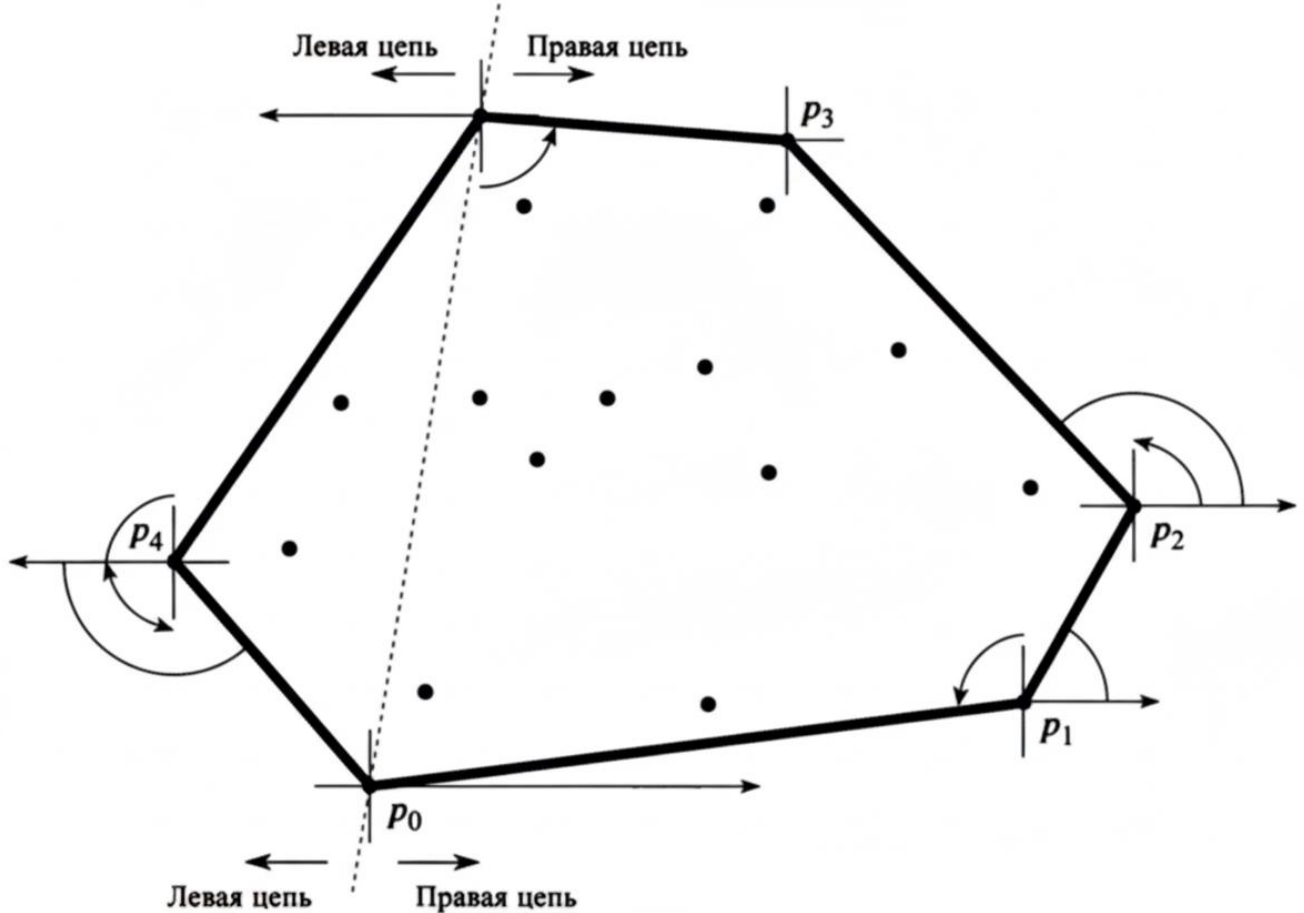
Код пример 18.3

Сканирование по Грэхему



Обход по Джарвису

При построении выпуклой оболочки множества точек Q путем ***обхода по Джарвису*** (Jarvis's march) используется метод, известный как ***упаковка пакета*** (package wrapping) или ***упаковка подарка*** (gift wrapping). Время выполнения алгоритма равно O(nh), где h - количество вершин CH(Q). В том случае, когда h равно O(lgn), обход по Джарвису работает быстрее, чем сканирование по Грэхему.



Обход по Джарвису. В качестве первой вершины выбирается наинизшая точка p_0. Следующая вершина, p_1, имеет наименьший полярный угол по отношению к p_0 среди всех точек.

Поиск пары ближайших точек

Подробности в лекции 16

время работы представленного алгоритма описывалось рекуррентным соотношением T(n) = 2T(n/2) + O(n), где T(n) - время работы алгоритма для n точек.

```
void SplitArray(Point *Y, int y_size, int x_median, Point **YL,
int *yl_size, Point **YR, int *yr_size) {
    *YL = (Point *)malloc(y_size * sizeof(Point));
    *YR = (Point *)malloc(y_size * sizeof(Point));
    *yl_size = 0;
    *yr_size = 0;
    for (int i = 0; i < y_size; i++) {
        if (is_in_PL(Y[i], x_median))
            (*YL)[(*yl_size)++] = Y[i];
        else
            (*YR)[(*yr_size)++] = Y[i];
    }
}
```

следует выполнить их предварительную сортировку (presorting), которая осуществляется один раз перед первым рекурсивным вызовом.